



Hochschule Aalen
UNIVERSITY OF APPLIED SCIENCES

Machine Learning & Deep Learning

05: COMPUTER VISION

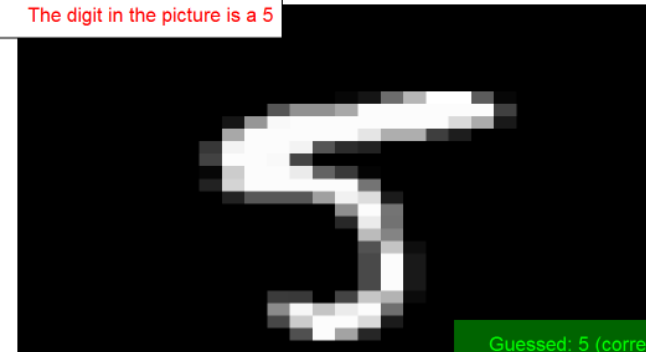
5. Mai 2026



Examples

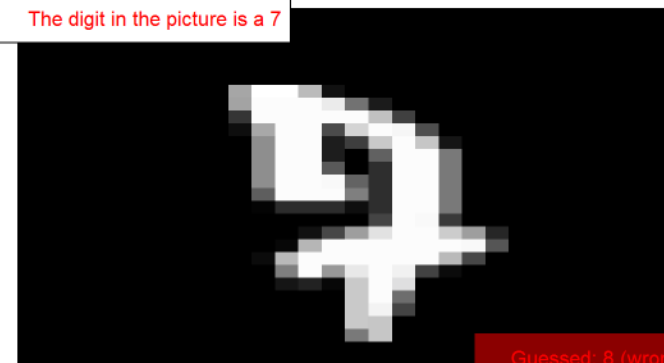


The digit in the picture is a 5



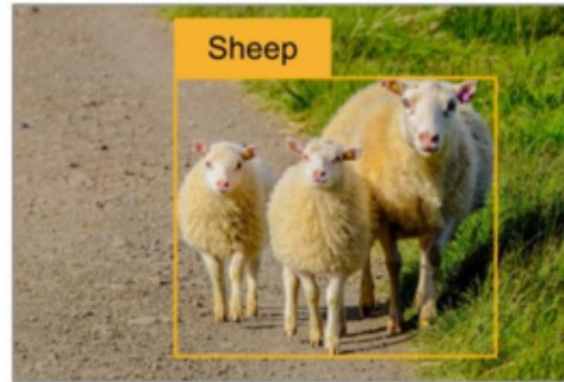
Guessed: 5 (correct)

The digit in the picture is a 7

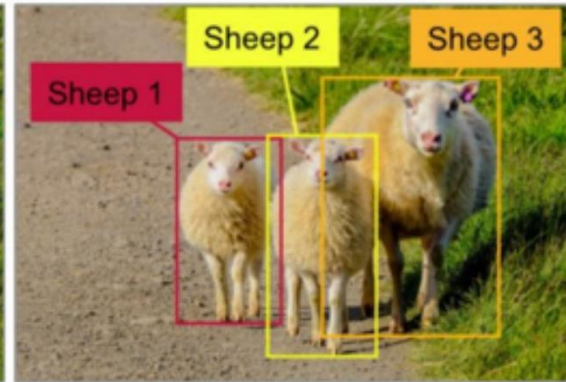


Guessed: 8 (wrong)

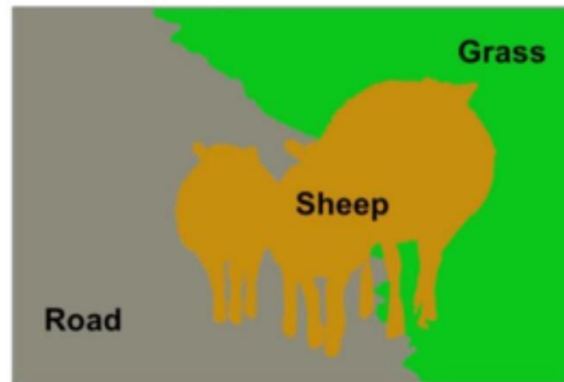
Tasks



Classification + Localization



Object Detection



Semantic Segmentation

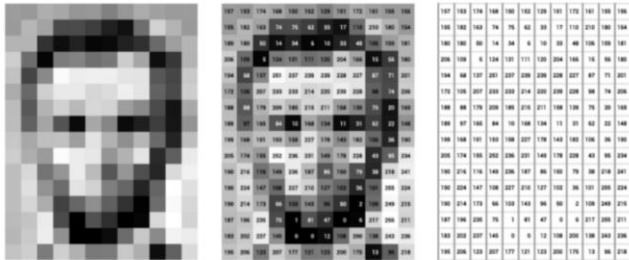


Instance Segmentation

Structure of images

Grayscale Images

Grayscale values have a single color channel. Notice that every grid position has a single value from 0–255 in the given image.

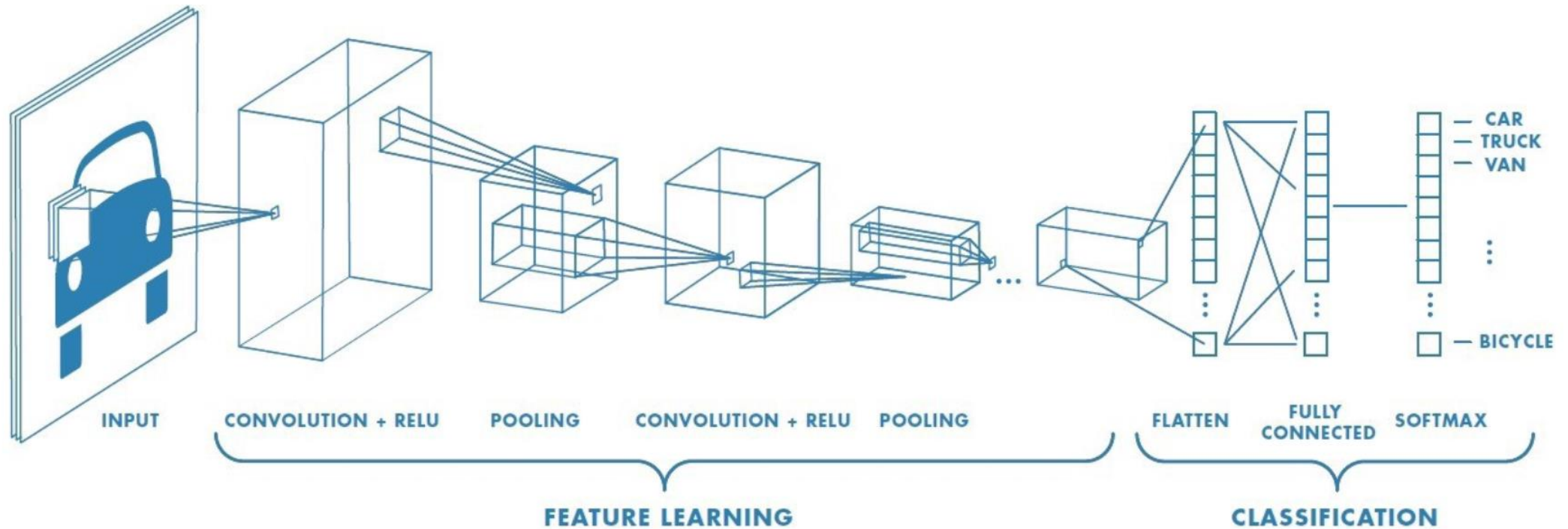


RGB Images

RGB images or colored images are represented by 3 color channels: red, green, and blue. They can be viewed 3 separate 2-dimensional grids that are stacked on top of each other, then merged.



Typical Structure



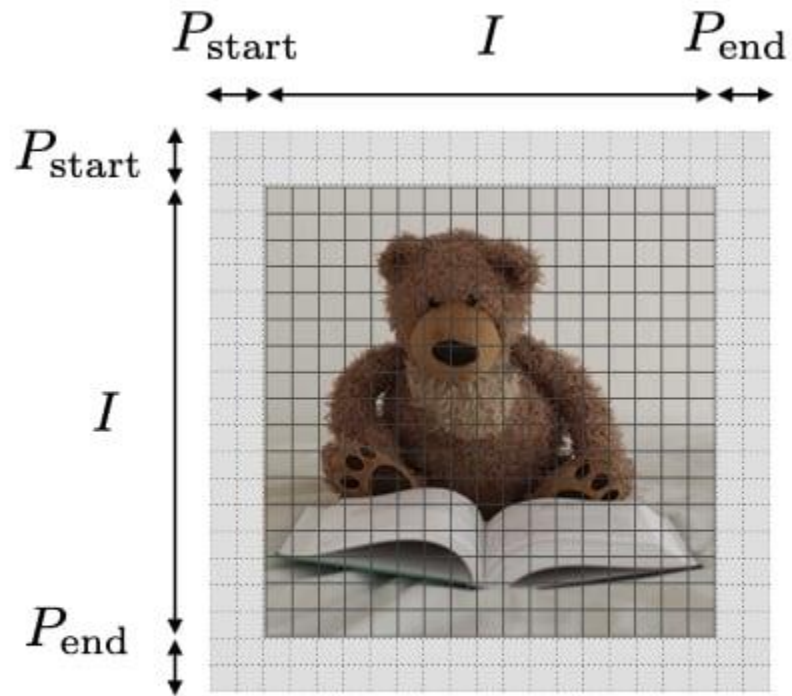
Basic Layer

- Convolution layer
 - Uses filters or kernels to find local patterns and features from the input image.
- Pooling layer
 - Downsamples the spatial dimensions of the input.
 - Reduces network's computational complexity.
 - Increases ability to recognize patterns regardless of scale/position.
- Fully Connected layer
 - Integrates extracted features into a traditional neural network.
 - Learns global relationships and generates task-specific outputs.

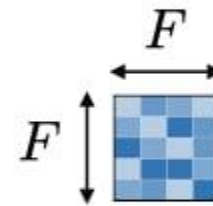
Convolution Layer

- Filters
 - Matrices to extract specific features from the input.
 - Each Filter is for a specific feature.
 - Values of filters are updated during training.
- Stride
 - Step size with which filter moves across input.
 - Larger strides → smaller output feature maps and faster computation.
- Padding
 - Control of spatial dimensions of the output by adding values around it.

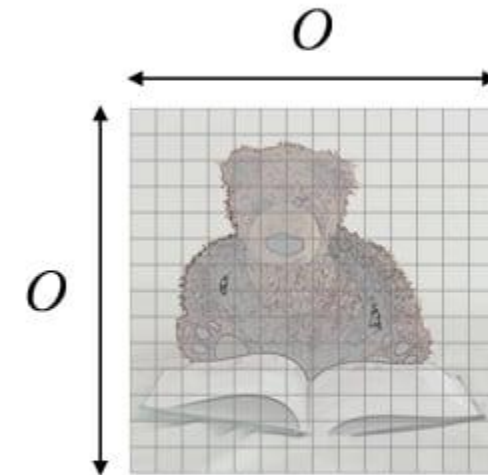
Convolution Layer



Input

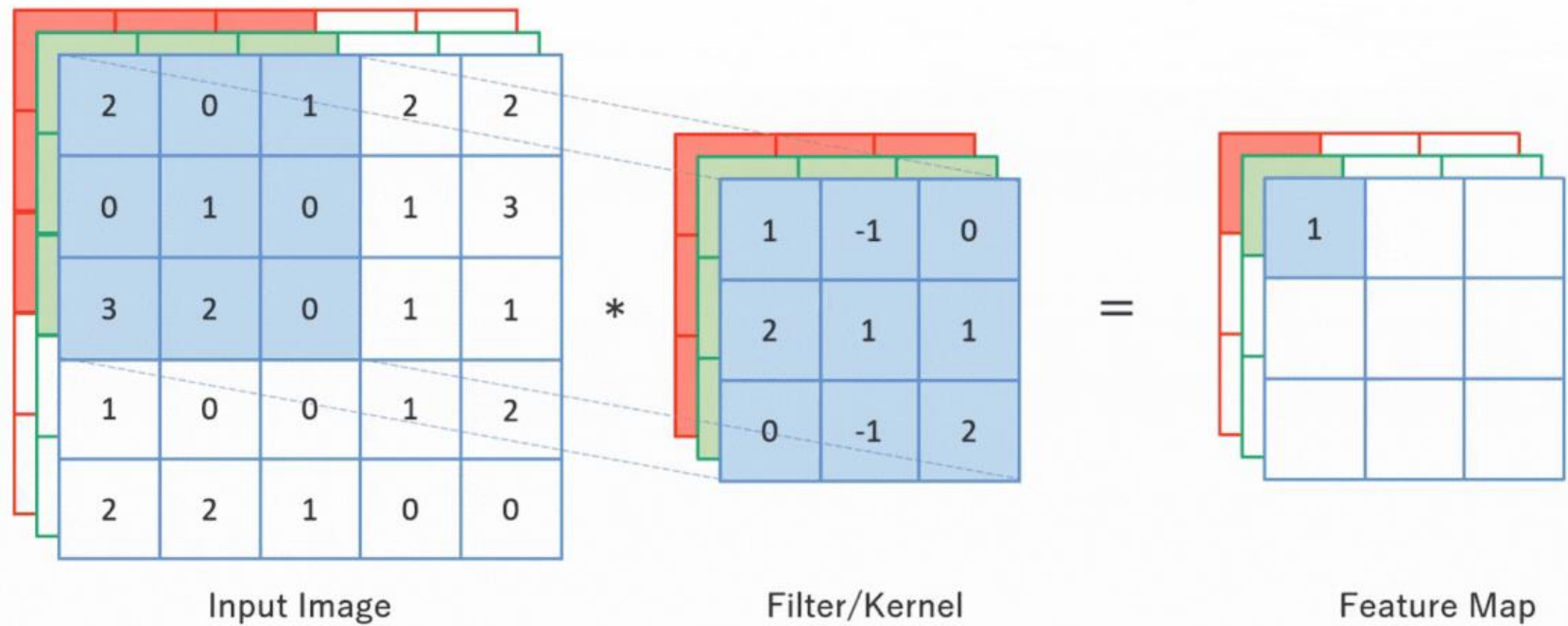


Filter



Output

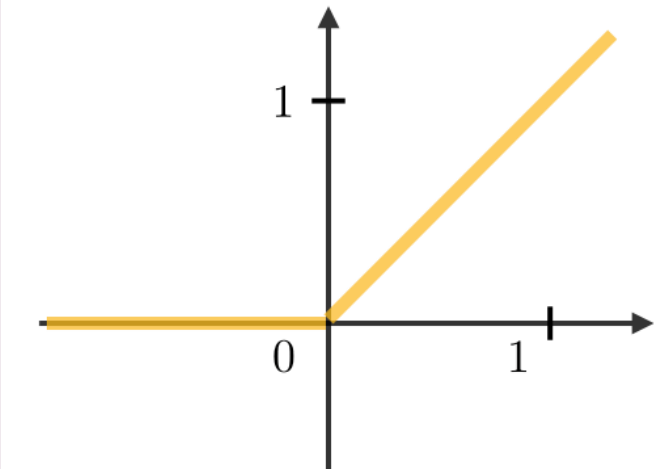
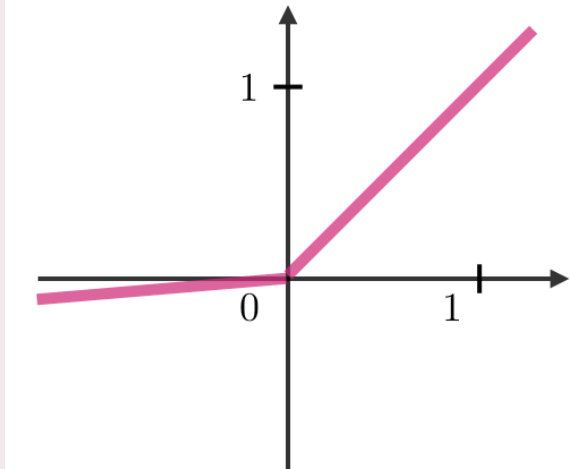
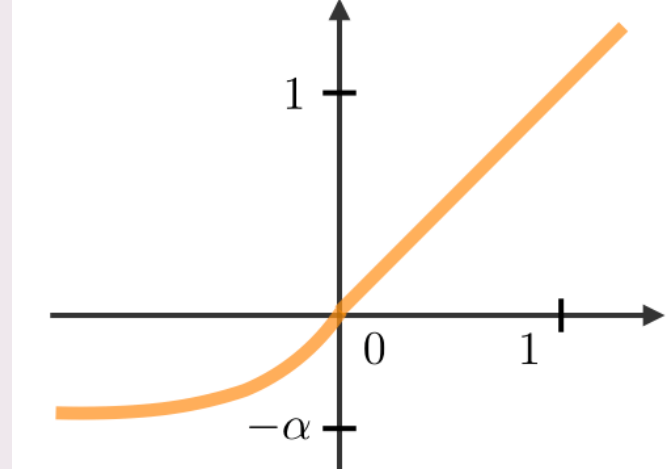
Convolution Layer



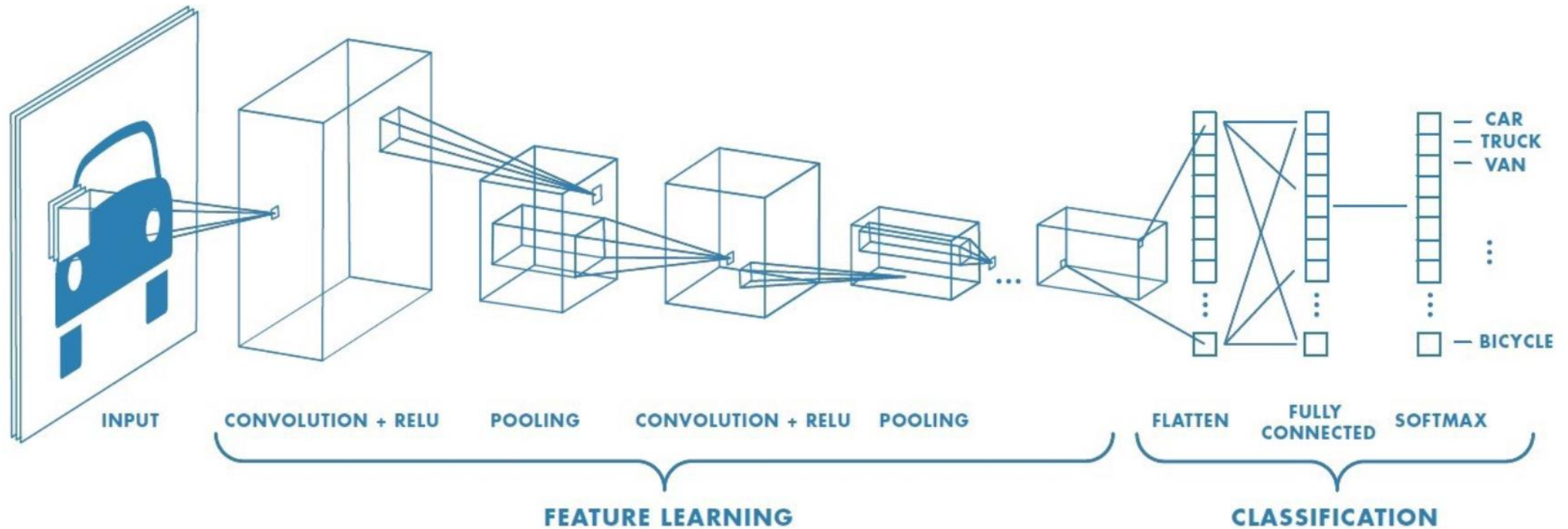
Convolution Layer

<table><tr><td>3₀</td><td>3₁</td><td>2₂</td><td>1</td><td>0</td></tr><tr><td>0₂</td><td>0₂</td><td>1₀</td><td>3</td><td>1</td></tr><tr><td>3₀</td><td>1₁</td><td>2₂</td><td>2</td><td>3</td></tr><tr><td>2</td><td>0</td><td>0</td><td>2</td><td>2</td></tr><tr><td>2</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table> <table><tr><td>12</td><td>12</td><td>17</td></tr><tr><td>10</td><td>17</td><td>19</td></tr><tr><td>9</td><td>6</td><td>14</td></tr></table>	3 ₀	3 ₁	2 ₂	1	0	0 ₂	0 ₂	1 ₀	3	1	3 ₀	1 ₁	2 ₂	2	3	2	0	0	2	2	2	0	0	0	1	12	12	17	10	17	19	9	6	14	<table><tr><td>3</td><td>3₀</td><td>2₁</td><td>1₂</td><td>0</td></tr><tr><td>0</td><td>0₂</td><td>1₂</td><td>3₀</td><td>1</td></tr><tr><td>3</td><td>1₀</td><td>2₁</td><td>2₂</td><td>3</td></tr><tr><td>2</td><td>0</td><td>0</td><td>2</td><td>2</td></tr><tr><td>2</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table> <table><tr><td>12</td><td>12</td><td>17</td></tr><tr><td>10</td><td>17</td><td>19</td></tr><tr><td>9</td><td>6</td><td>14</td></tr></table>	3	3 ₀	2 ₁	1 ₂	0	0	0 ₂	1 ₂	3 ₀	1	3	1 ₀	2 ₁	2 ₂	3	2	0	0	2	2	2	0	0	0	1	12	12	17	10	17	19	9	6	14	<table><tr><td>3</td><td>3</td><td>2₀</td><td>1₁</td><td>0₂</td></tr><tr><td>0</td><td>0</td><td>1₂</td><td>3₂</td><td>1₀</td></tr><tr><td>3</td><td>1</td><td>2₀</td><td>2₁</td><td>3₂</td></tr><tr><td>2</td><td>0</td><td>0</td><td>2</td><td>2</td></tr><tr><td>2</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table> <table><tr><td>12</td><td>12</td><td>17</td></tr><tr><td>10</td><td>17</td><td>19</td></tr><tr><td>9</td><td>6</td><td>14</td></tr></table>	3	3	2 ₀	1 ₁	0 ₂	0	0	1 ₂	3 ₂	1 ₀	3	1	2 ₀	2 ₁	3 ₂	2	0	0	2	2	2	0	0	0	1	12	12	17	10	17	19	9	6	14
3 ₀	3 ₁	2 ₂	1	0																																																																																																				
0 ₂	0 ₂	1 ₀	3	1																																																																																																				
3 ₀	1 ₁	2 ₂	2	3																																																																																																				
2	0	0	2	2																																																																																																				
2	0	0	0	1																																																																																																				
12	12	17																																																																																																						
10	17	19																																																																																																						
9	6	14																																																																																																						
3	3 ₀	2 ₁	1 ₂	0																																																																																																				
0	0 ₂	1 ₂	3 ₀	1																																																																																																				
3	1 ₀	2 ₁	2 ₂	3																																																																																																				
2	0	0	2	2																																																																																																				
2	0	0	0	1																																																																																																				
12	12	17																																																																																																						
10	17	19																																																																																																						
9	6	14																																																																																																						
3	3	2 ₀	1 ₁	0 ₂																																																																																																				
0	0	1 ₂	3 ₂	1 ₀																																																																																																				
3	1	2 ₀	2 ₁	3 ₂																																																																																																				
2	0	0	2	2																																																																																																				
2	0	0	0	1																																																																																																				
12	12	17																																																																																																						
10	17	19																																																																																																						
9	6	14																																																																																																						
<table><tr><td>3</td><td>3</td><td>2</td><td>1</td><td>0</td></tr><tr><td>0₀</td><td>0₁</td><td>1₂</td><td>3</td><td>1</td></tr><tr><td>3₂</td><td>1₂</td><td>2₀</td><td>2</td><td>3</td></tr><tr><td>2₀</td><td>0₁</td><td>0₂</td><td>2</td><td>2</td></tr><tr><td>2</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table> <table><tr><td>12</td><td>12</td><td>17</td></tr><tr><td>10</td><td>17</td><td>19</td></tr><tr><td>9</td><td>6</td><td>14</td></tr></table>	3	3	2	1	0	0 ₀	0 ₁	1 ₂	3	1	3 ₂	1 ₂	2 ₀	2	3	2 ₀	0 ₁	0 ₂	2	2	2	0	0	0	1	12	12	17	10	17	19	9	6	14	<table><tr><td>3</td><td>3</td><td>2</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0₀</td><td>1₁</td><td>3₂</td><td>1</td></tr><tr><td>3</td><td>1₂</td><td>2₂</td><td>2₀</td><td>3</td></tr><tr><td>2</td><td>0₀</td><td>0₁</td><td>2₂</td><td>2</td></tr><tr><td>2</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table> <table><tr><td>12</td><td>12</td><td>17</td></tr><tr><td>10</td><td>17</td><td>19</td></tr><tr><td>9</td><td>6</td><td>14</td></tr></table>	3	3	2	1	0	0	0 ₀	1 ₁	3 ₂	1	3	1 ₂	2 ₂	2 ₀	3	2	0 ₀	0 ₁	2 ₂	2	2	0	0	0	1	12	12	17	10	17	19	9	6	14	<table><tr><td>3</td><td>3</td><td>2</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1₀</td><td>3₁</td><td>1₂</td></tr><tr><td>3</td><td>1</td><td>2₂</td><td>2₂</td><td>3₀</td></tr><tr><td>2</td><td>0</td><td>0₀</td><td>2₁</td><td>2₂</td></tr><tr><td>2</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table> <table><tr><td>12</td><td>12</td><td>17</td></tr><tr><td>10</td><td>17</td><td>19</td></tr><tr><td>9</td><td>6</td><td>14</td></tr></table>	3	3	2	1	0	0	0	1 ₀	3 ₁	1 ₂	3	1	2 ₂	2 ₂	3 ₀	2	0	0 ₀	2 ₁	2 ₂	2	0	0	0	1	12	12	17	10	17	19	9	6	14
3	3	2	1	0																																																																																																				
0 ₀	0 ₁	1 ₂	3	1																																																																																																				
3 ₂	1 ₂	2 ₀	2	3																																																																																																				
2 ₀	0 ₁	0 ₂	2	2																																																																																																				
2	0	0	0	1																																																																																																				
12	12	17																																																																																																						
10	17	19																																																																																																						
9	6	14																																																																																																						
3	3	2	1	0																																																																																																				
0	0 ₀	1 ₁	3 ₂	1																																																																																																				
3	1 ₂	2 ₂	2 ₀	3																																																																																																				
2	0 ₀	0 ₁	2 ₂	2																																																																																																				
2	0	0	0	1																																																																																																				
12	12	17																																																																																																						
10	17	19																																																																																																						
9	6	14																																																																																																						
3	3	2	1	0																																																																																																				
0	0	1 ₀	3 ₁	1 ₂																																																																																																				
3	1	2 ₂	2 ₂	3 ₀																																																																																																				
2	0	0 ₀	2 ₁	2 ₂																																																																																																				
2	0	0	0	1																																																																																																				
12	12	17																																																																																																						
10	17	19																																																																																																						
9	6	14																																																																																																						
<table><tr><td>3</td><td>3</td><td>2</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>3</td><td>1</td></tr><tr><td>3₀</td><td>1₁</td><td>2₂</td><td>2</td><td>3</td></tr><tr><td>2₂</td><td>0₂</td><td>0₀</td><td>2</td><td>2</td></tr><tr><td>2₀</td><td>0₁</td><td>0₂</td><td>0</td><td>1</td></tr></table> <table><tr><td>12</td><td>12</td><td>17</td></tr><tr><td>10</td><td>17</td><td>19</td></tr><tr><td>9</td><td>6</td><td>14</td></tr></table>	3	3	2	1	0	0	0	1	3	1	3 ₀	1 ₁	2 ₂	2	3	2 ₂	0 ₂	0 ₀	2	2	2 ₀	0 ₁	0 ₂	0	1	12	12	17	10	17	19	9	6	14	<table><tr><td>3</td><td>3</td><td>2</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>3</td><td>1</td></tr><tr><td>3</td><td>1₀</td><td>2₁</td><td>2₂</td><td>3</td></tr><tr><td>2</td><td>0₂</td><td>0₂</td><td>2₀</td><td>2</td></tr><tr><td>2</td><td>0₀</td><td>0₁</td><td>0₂</td><td>1</td></tr></table> <table><tr><td>12</td><td>12</td><td>17</td></tr><tr><td>10</td><td>17</td><td>19</td></tr><tr><td>9</td><td>6</td><td>14</td></tr></table>	3	3	2	1	0	0	0	1	3	1	3	1 ₀	2 ₁	2 ₂	3	2	0 ₂	0 ₂	2 ₀	2	2	0 ₀	0 ₁	0 ₂	1	12	12	17	10	17	19	9	6	14	<table><tr><td>3</td><td>3</td><td>2</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>3</td><td>1</td></tr><tr><td>3</td><td>1</td><td>2₀</td><td>2₁</td><td>3₂</td></tr><tr><td>2</td><td>0</td><td>0₂</td><td>2₂</td><td>2₀</td></tr><tr><td>2</td><td>0</td><td>0₀</td><td>0₁</td><td>1₂</td></tr></table> <table><tr><td>12</td><td>12</td><td>17</td></tr><tr><td>10</td><td>17</td><td>19</td></tr><tr><td>9</td><td>6</td><td>14</td></tr></table>	3	3	2	1	0	0	0	1	3	1	3	1	2 ₀	2 ₁	3 ₂	2	0	0 ₂	2 ₂	2 ₀	2	0	0 ₀	0 ₁	1 ₂	12	12	17	10	17	19	9	6	14
3	3	2	1	0																																																																																																				
0	0	1	3	1																																																																																																				
3 ₀	1 ₁	2 ₂	2	3																																																																																																				
2 ₂	0 ₂	0 ₀	2	2																																																																																																				
2 ₀	0 ₁	0 ₂	0	1																																																																																																				
12	12	17																																																																																																						
10	17	19																																																																																																						
9	6	14																																																																																																						
3	3	2	1	0																																																																																																				
0	0	1	3	1																																																																																																				
3	1 ₀	2 ₁	2 ₂	3																																																																																																				
2	0 ₂	0 ₂	2 ₀	2																																																																																																				
2	0 ₀	0 ₁	0 ₂	1																																																																																																				
12	12	17																																																																																																						
10	17	19																																																																																																						
9	6	14																																																																																																						
3	3	2	1	0																																																																																																				
0	0	1	3	1																																																																																																				
3	1	2 ₀	2 ₁	3 ₂																																																																																																				
2	0	0 ₂	2 ₂	2 ₀																																																																																																				
2	0	0 ₀	0 ₁	1 ₂																																																																																																				
12	12	17																																																																																																						
10	17	19																																																																																																						
9	6	14																																																																																																						

Convolution Layer + Relu

ReLU	Leaky ReLU	ELU
$g(z) = \max(0, z)$	$g(z) = \max(\epsilon z, z)$ with $\epsilon \ll 1$	$g(z) = \max(\alpha(ez - 1), z)$ with $\alpha \ll 1$
		
Non-linearity complexities biologically interpretable	Addresses dying ReLU issue for negative values	Differentiable everywhere

Typical Structure



Pooling Layer

- Purpose
 - Reduces spatial dimensions → fewer parameters and lower computation.
 - Provides translation invariance, small shifts in the input don't affect results.
 - Helps prevent overfitting by summarizing activations.
- Stride
 - Define how far the pooling window moves.
 - Larger strides → greater downsampling (smaller output feature map).
- Padding
 - Control of spatial dimensions of the output by adding values around it.
- Types
 - Max Pooling: takes largest value from each window (captures strongest feature).
 - Average Pooling: takes the mean value from each window (smooths representation).

Pooling Layer

Max Pooling

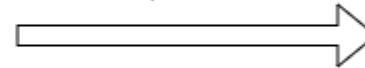
<table><tr><td>3</td><td>3</td><td>2</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>3</td><td>1</td></tr><tr><td>3</td><td>1</td><td>2</td><td>2</td><td>3</td></tr><tr><td>2</td><td>0</td><td>0</td><td>2</td><td>2</td></tr><tr><td>2</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	3	3	2	1	0	0	0	1	3	1	3	1	2	2	3	2	0	0	2	2	2	0	0	0	1	<table><tr><td>3</td><td>3</td><td>3</td></tr><tr><td>3</td><td>3</td><td>3</td></tr><tr><td>3</td><td>2</td><td>3</td></tr></table>	3	3	3	3	3	3	3	2	3	<table><tr><td>3</td><td>3</td><td>2</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>3</td><td>1</td></tr><tr><td>3</td><td>1</td><td>2</td><td>2</td><td>3</td></tr><tr><td>2</td><td>0</td><td>0</td><td>2</td><td>2</td></tr><tr><td>2</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	3	3	2	1	0	0	0	1	3	1	3	1	2	2	3	2	0	0	2	2	2	0	0	0	1	<table><tr><td>3</td><td>3</td><td>3</td></tr><tr><td>3</td><td>3</td><td>3</td></tr><tr><td>3</td><td>2</td><td>3</td></tr></table>	3	3	3	3	3	3	3	2	3	<table><tr><td>3</td><td>3</td><td>2</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>3</td><td>1</td></tr><tr><td>3</td><td>1</td><td>2</td><td>2</td><td>3</td></tr><tr><td>2</td><td>0</td><td>0</td><td>2</td><td>2</td></tr><tr><td>2</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	3	3	2	1	0	0	0	1	3	1	3	1	2	2	3	2	0	0	2	2	2	0	0	0	1	<table><tr><td>3</td><td>3</td><td>3</td></tr><tr><td>3</td><td>3</td><td>3</td></tr><tr><td>3</td><td>2</td><td>3</td></tr></table>	3	3	3	3	3	3	3	2	3
3	3	2	1	0																																																																																																							
0	0	1	3	1																																																																																																							
3	1	2	2	3																																																																																																							
2	0	0	2	2																																																																																																							
2	0	0	0	1																																																																																																							
3	3	3																																																																																																									
3	3	3																																																																																																									
3	2	3																																																																																																									
3	3	2	1	0																																																																																																							
0	0	1	3	1																																																																																																							
3	1	2	2	3																																																																																																							
2	0	0	2	2																																																																																																							
2	0	0	0	1																																																																																																							
3	3	3																																																																																																									
3	3	3																																																																																																									
3	2	3																																																																																																									
3	3	2	1	0																																																																																																							
0	0	1	3	1																																																																																																							
3	1	2	2	3																																																																																																							
2	0	0	2	2																																																																																																							
2	0	0	0	1																																																																																																							
3	3	3																																																																																																									
3	3	3																																																																																																									
3	2	3																																																																																																									
<table><tr><td>3</td><td>3</td><td>2</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>3</td><td>1</td></tr><tr><td>3</td><td>1</td><td>2</td><td>2</td><td>3</td></tr><tr><td>2</td><td>0</td><td>0</td><td>2</td><td>2</td></tr><tr><td>2</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	3	3	2	1	0	0	0	1	3	1	3	1	2	2	3	2	0	0	2	2	2	0	0	0	1	<table><tr><td>3</td><td>3</td><td>3</td></tr><tr><td>3</td><td>3</td><td>3</td></tr><tr><td>3</td><td>2</td><td>3</td></tr></table>	3	3	3	3	3	3	3	2	3	<table><tr><td>3</td><td>3</td><td>2</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>3</td><td>1</td></tr><tr><td>3</td><td>1</td><td>2</td><td>2</td><td>3</td></tr><tr><td>2</td><td>0</td><td>0</td><td>2</td><td>2</td></tr><tr><td>2</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	3	3	2	1	0	0	0	1	3	1	3	1	2	2	3	2	0	0	2	2	2	0	0	0	1	<table><tr><td>3</td><td>3</td><td>3</td></tr><tr><td>3</td><td>3</td><td>3</td></tr><tr><td>3</td><td>2</td><td>3</td></tr></table>	3	3	3	3	3	3	3	2	3	<table><tr><td>3</td><td>3</td><td>2</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>3</td><td>1</td></tr><tr><td>3</td><td>1</td><td>2</td><td>2</td><td>3</td></tr><tr><td>2</td><td>0</td><td>0</td><td>2</td><td>2</td></tr><tr><td>2</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	3	3	2	1	0	0	0	1	3	1	3	1	2	2	3	2	0	0	2	2	2	0	0	0	1	<table><tr><td>3</td><td>3</td><td>3</td></tr><tr><td>3</td><td>3</td><td>3</td></tr><tr><td>3</td><td>2</td><td>3</td></tr></table>	3	3	3	3	3	3	3	2	3
3	3	2	1	0																																																																																																							
0	0	1	3	1																																																																																																							
3	1	2	2	3																																																																																																							
2	0	0	2	2																																																																																																							
2	0	0	0	1																																																																																																							
3	3	3																																																																																																									
3	3	3																																																																																																									
3	2	3																																																																																																									
3	3	2	1	0																																																																																																							
0	0	1	3	1																																																																																																							
3	1	2	2	3																																																																																																							
2	0	0	2	2																																																																																																							
2	0	0	0	1																																																																																																							
3	3	3																																																																																																									
3	3	3																																																																																																									
3	2	3																																																																																																									
3	3	2	1	0																																																																																																							
0	0	1	3	1																																																																																																							
3	1	2	2	3																																																																																																							
2	0	0	2	2																																																																																																							
2	0	0	0	1																																																																																																							
3	3	3																																																																																																									
3	3	3																																																																																																									
3	2	3																																																																																																									
<table><tr><td>3</td><td>3</td><td>2</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>3</td><td>1</td></tr><tr><td>3</td><td>1</td><td>2</td><td>2</td><td>3</td></tr><tr><td>2</td><td>0</td><td>0</td><td>2</td><td>2</td></tr><tr><td>2</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	3	3	2	1	0	0	0	1	3	1	3	1	2	2	3	2	0	0	2	2	2	0	0	0	1	<table><tr><td>3</td><td>3</td><td>3</td></tr><tr><td>3</td><td>3</td><td>3</td></tr><tr><td>3</td><td>2</td><td>3</td></tr></table>	3	3	3	3	3	3	3	2	3	<table><tr><td>3</td><td>3</td><td>2</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>3</td><td>1</td></tr><tr><td>3</td><td>1</td><td>2</td><td>2</td><td>3</td></tr><tr><td>2</td><td>0</td><td>0</td><td>2</td><td>2</td></tr><tr><td>2</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	3	3	2	1	0	0	0	1	3	1	3	1	2	2	3	2	0	0	2	2	2	0	0	0	1	<table><tr><td>3</td><td>3</td><td>3</td></tr><tr><td>3</td><td>3</td><td>3</td></tr><tr><td>3</td><td>2</td><td>3</td></tr></table>	3	3	3	3	3	3	3	2	3	<table><tr><td>3</td><td>3</td><td>2</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>3</td><td>1</td></tr><tr><td>3</td><td>1</td><td>2</td><td>2</td><td>3</td></tr><tr><td>2</td><td>0</td><td>0</td><td>2</td><td>2</td></tr><tr><td>2</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	3	3	2	1	0	0	0	1	3	1	3	1	2	2	3	2	0	0	2	2	2	0	0	0	1	<table><tr><td>3</td><td>3</td><td>3</td></tr><tr><td>3</td><td>3</td><td>3</td></tr><tr><td>3</td><td>2</td><td>3</td></tr></table>	3	3	3	3	3	3	3	2	3
3	3	2	1	0																																																																																																							
0	0	1	3	1																																																																																																							
3	1	2	2	3																																																																																																							
2	0	0	2	2																																																																																																							
2	0	0	0	1																																																																																																							
3	3	3																																																																																																									
3	3	3																																																																																																									
3	2	3																																																																																																									
3	3	2	1	0																																																																																																							
0	0	1	3	1																																																																																																							
3	1	2	2	3																																																																																																							
2	0	0	2	2																																																																																																							
2	0	0	0	1																																																																																																							
3	3	3																																																																																																									
3	3	3																																																																																																									
3	2	3																																																																																																									
3	3	2	1	0																																																																																																							
0	0	1	3	1																																																																																																							
3	1	2	2	3																																																																																																							
2	0	0	2	2																																																																																																							
2	0	0	0	1																																																																																																							
3	3	3																																																																																																									
3	3	3																																																																																																									
3	2	3																																																																																																									

Pooling Layer

Average Pooling

1	4	2	7
2	6	8	5
3	4	0	7
1	2	3	1

Average Pooling
2x2, Stride 2



3.25	5.50
2.50	2.75

Pooling Layer

max pooling

(446, 450)



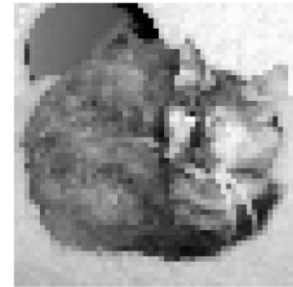
(223, 225)



(111, 112)



(55, 56)



average pooling

(446, 450)



(223, 225)



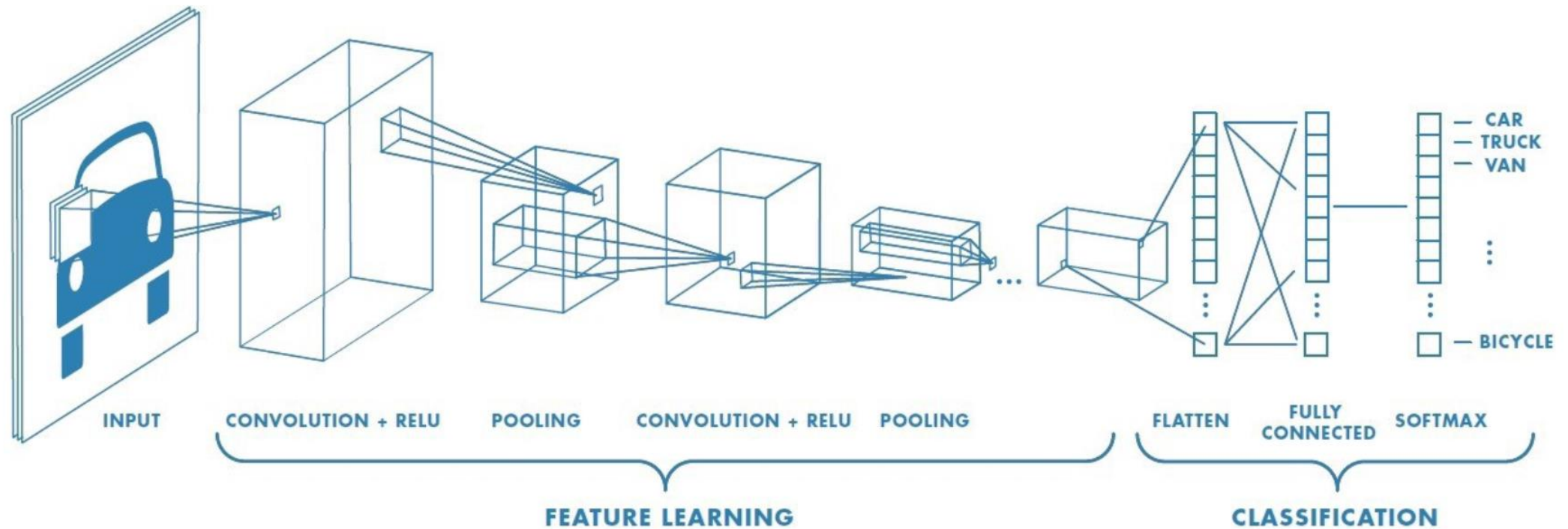
(111, 112)



(55, 56)



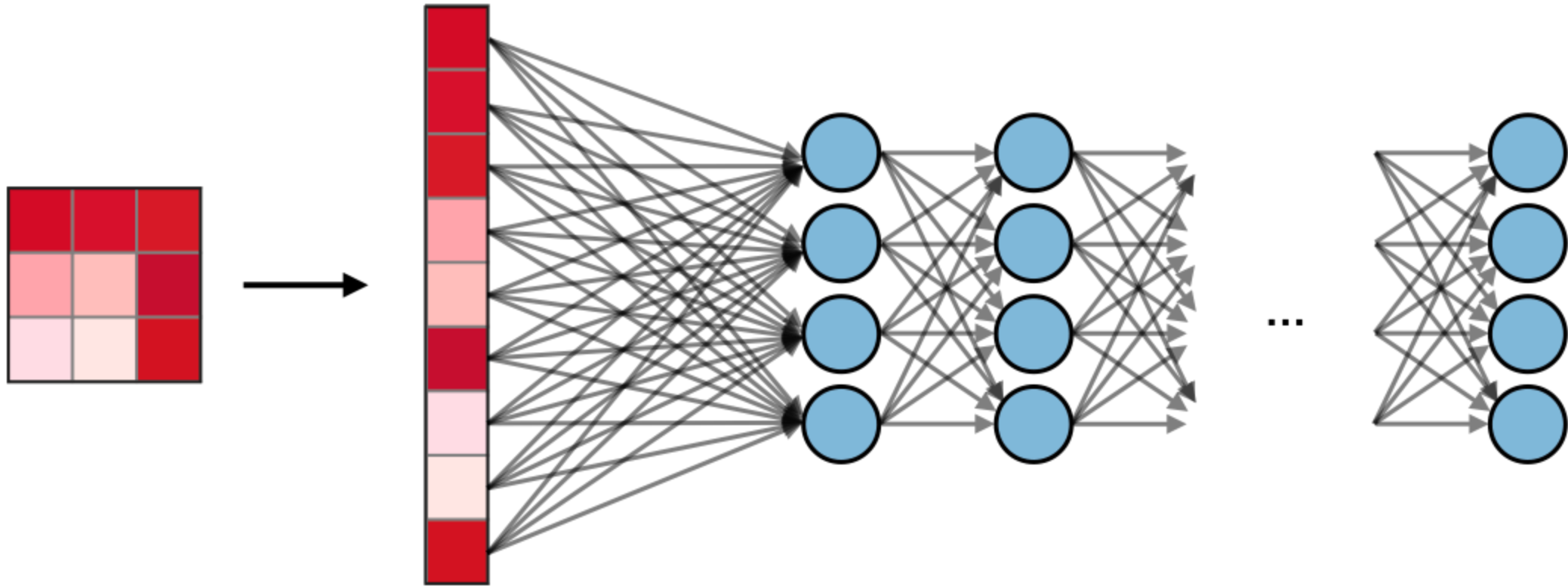
Typical Structure



Fully Connected Layer

- Fully Connected (Dense) layers combine information from all extracted features to make the final decision.
- Connections
 - Every neuron in one layer is connected to all neurons in the next layer.
 - Enables global reasoning across the entire input.
 - Used in classification to map high-level features to class scores.
- Purpose
 - Transforms learned spatial features into class probabilities or regression outputs.
 - Captures nonlinear combinations of visual patterns.
- Notes
 - Requires many parameters, can lead to high memory cost.
 - Modern architectures often replace large FC layers with Global Average Pooling to improve efficiency.

Fully Connected Layer



CNN - Example

Basic CNN for MNIST digit classification

- Input image: $1 \times 28 \times 28$
 - One grayscale image with 28×28 pixels
- Conv1: $1 \rightarrow 32$ channels
 - Learns 32 filters and creates 32 feature maps
- ReLU activation
 - Keeps positive values and sets negative values to 0
- Max Pooling: 2×2 , stride 2
 - Reduces spatial size from 28×28 to 14×14

```
class BasicCNN(nn.Module):  
  
    def __init__(self):  
        super().__init__()  
        # Input: 1 x 28 x 28  
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=32,  
                                kernel_size=3, stride=1, padding=1)  
  
        # Output: 32 x 28 x 28  
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)  
  
        # After pooling: 32 x 14 x 14  
        self.conv2 = nn.Conv2d(in_channels=32, out_channels=64,  
                                kernel_size=3, stride=1, padding=1)  
  
        # Output: 64 x 14 x 1  
        # After pooling: 64 x 7 x 7  
        self.fc1 = nn.Linear(64 * 7 * 7, 128)  
        self.fc2 = nn.Linear(128, 10) # 10 classes: digits 0-9  
  
    def forward(self, x):  
        x = self.conv1(x) # Convolution  
        x = F.relu(x) # ReLU activation  
        x = self.pool(x) # Max pooling  
        x = self.conv2(x) # Convolution  
        x = F.relu(x) # ReLU activation  
        x = self.pool(x) # Max pooling  
        x = torch.flatten(x, 1) # Keep batch dimension, flatten the rest  
        x = self.fc1(x) # Fully connected layer  
        x = F.relu(x) # ReLU activation  
        x = self.fc2(x) # Output layer  
        return x
```

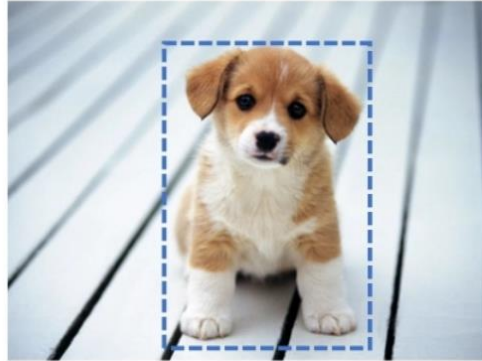
CNN - Example

- Conv2: 32 → 64 channels
 - Learns more complex features from the first feature maps
- Max Pooling again
 - Reduces spatial size from 14×14 to 7×7
- Flatten
 - Converts $64 \times 7 \times 7$ feature maps into a vector of 3136 values
- Fully connected layers
 - Map extracted features to 10 output classes
- Output: 10 logits
 - One score for each digit from 0 to 9

```
class BasicCNN(nn.Module):  
  
    def __init__(self):  
        super().__init__()  
        # Input: 1 x 28 x 28  
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=32,  
                                kernel_size=3, stride=1, padding=1)  
  
        # Output: 32 x 28 x 28  
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)  
  
        # After pooling: 32 x 14 x 14  
        self.conv2 = nn.Conv2d(in_channels=32, out_channels=64,  
                                kernel_size=3, stride=1, padding=1)  
  
        # Output: 64 x 14 x 1  
        # After pooling: 64 x 7 x 7  
        self.fc1 = nn.Linear(64 * 7 * 7, 128)  
        self.fc2 = nn.Linear(128, 10) # 10 classes: digits 0-9  
  
    def forward(self, x):  
        x = self.conv1(x) # Convolution  
        x = F.relu(x) # ReLU activation  
        x = self.pool(x) # Max pooling  
        x = self.conv2(x) # Convolution  
        x = F.relu(x) # ReLU activation  
        x = self.pool(x) # Max pooling  
        x = torch.flatten(x, 1) # Keep batch dimension, flatten the rest  
        x = self.fc1(x) # Fully connected layer  
        x = F.relu(x) # ReLU activation  
        x = self.fc2(x) # Output layer  
        return x
```

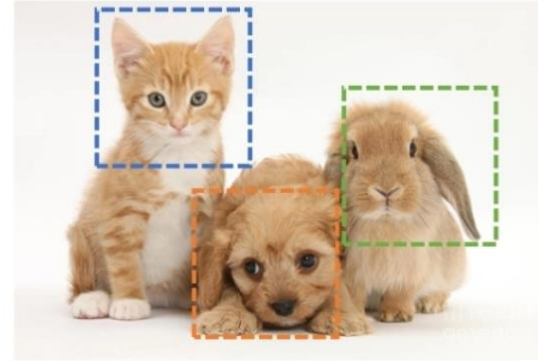
Object Detection

★ Single-label classification



Dog

★ Multi-label classification



Cat, Dog, Rabbit

Goal:

- Determine for a given image:
 - Which objects are in the image
 - Where are these objects
 - Confidence score of detection

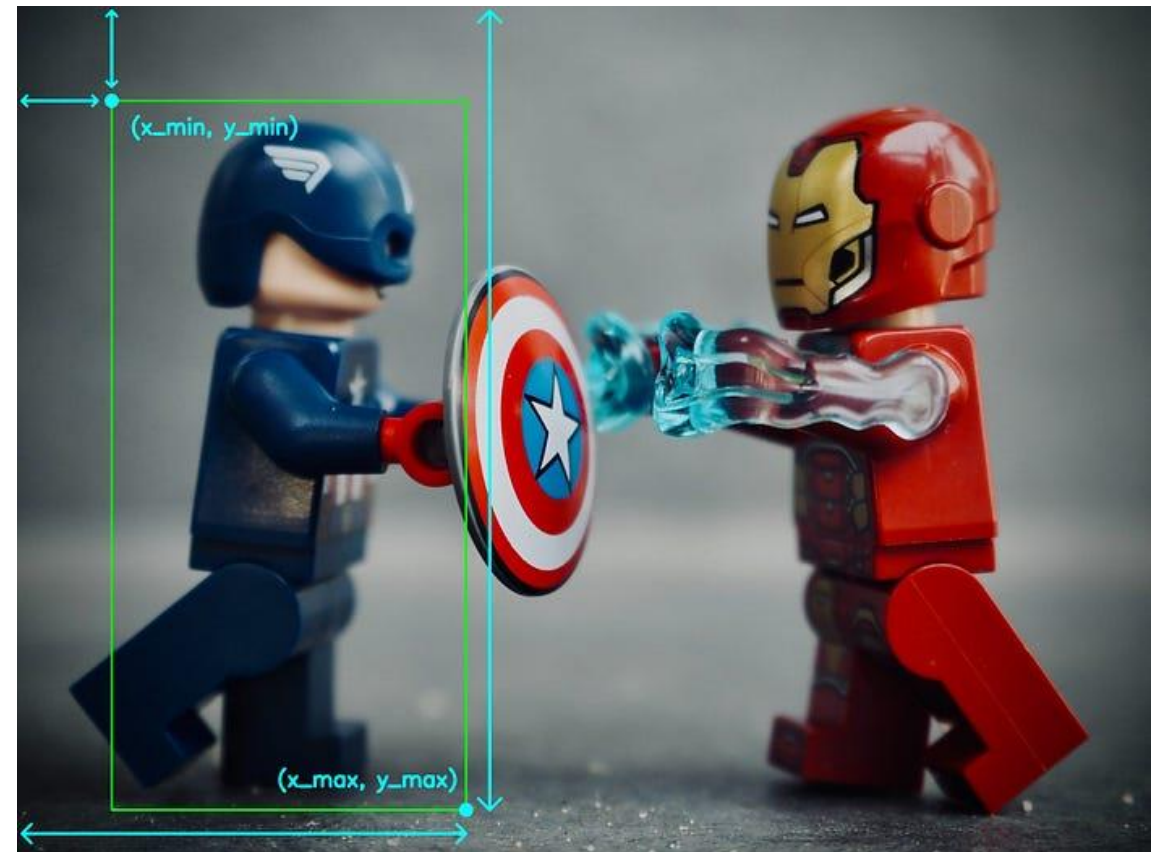
Object Detection

- Identifying and locating multiple objects within an image or video.
- Image Classification:
 - Assigns a label to an entire image. Doesn't provide spatial information.
- Object Localization:
 - Identifies the object and determines its position within the image. Involves drawing bounding boxes around the object.
- Object Detection:
 - Merges image classification and localization. Detects multiple objects in an image, assigns labels and provides locations through bounding boxes.

Object Detection

Bounding Boxes

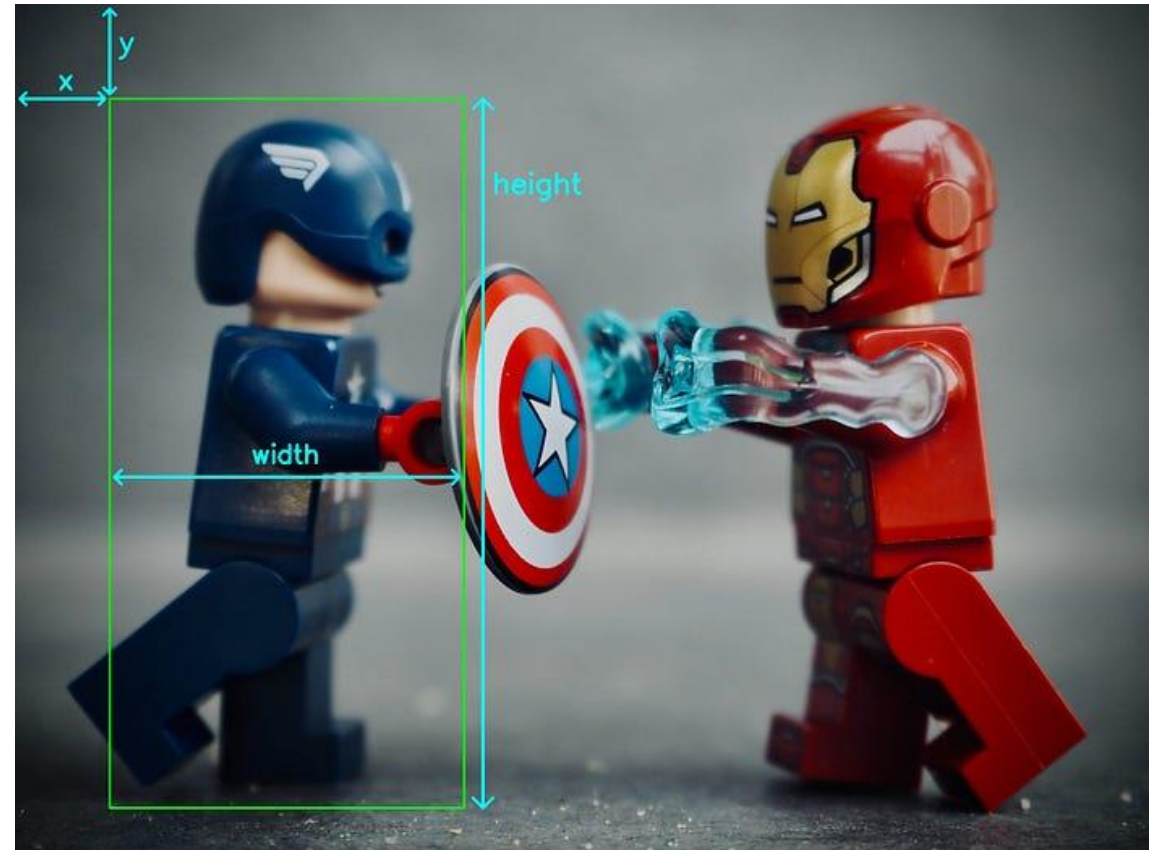
- Rectangle, drawn on an image to highlight the presence of an object of interest at spatial location.
- Pascal VOC format:
 - XYXY / corner format
(x_min, y_min, x_max, y_max)



Object Detection

Bounding Boxes

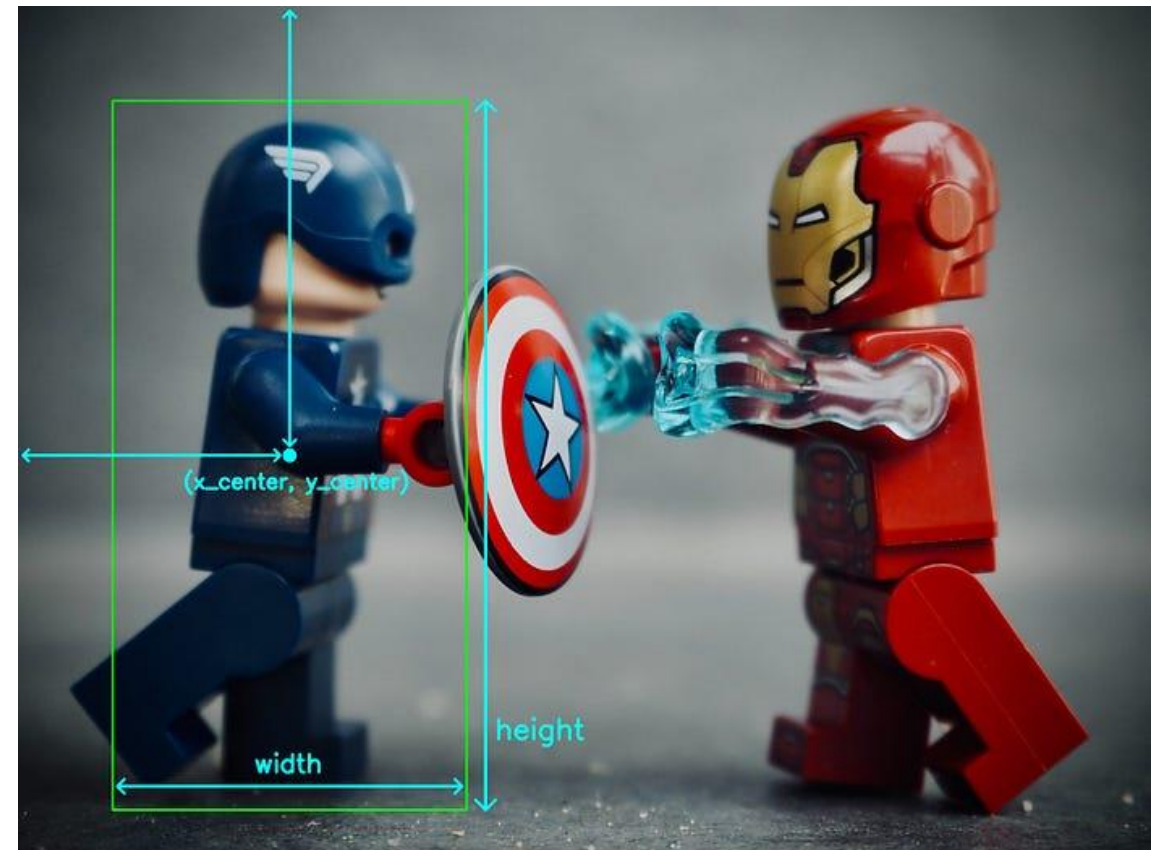
- Rectangle, drawn on an image to highlight the presence of an object of interest at spatial location.
- COCO format:
 - XYWH / top-left format
(x_min, y_min, width, height)



Object Detection

Bounding Boxes

- Rectangle, drawn on an image to highlight the presence of an object of interest at spatial location.
- YOLO format:
 - CXCXYWH / (normalized) center format
(x_center, y_center, width, height)
normalized between 0 and 1



Object Detection Metrics

Measures how much predicted bounding box overlaps with ground truth bounding box.


$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

- Threshold-based metric to determine if predicted box counts as correct detection (true positive)
- Higher IoU → better localization accuracy

Threshold of 50%

Object Detection

Metrics

Exercise: Calculate the Intersection over Union for the following two scenarios and identify whether it is a correct detection or not. All bounding boxes are written in the following format: [x, y, width, height]

- Steps: Calculate Intersection Box; Compute Intersection Area; Compute Union area; Compute IoU
- Scenario 1:
 - Ground Truth Box: [2, 2, 4, 4]
 - Predicted Box: [4, 4, 4, 4]
- Scenario 2:
 - Ground Truth Box: [2, 2, 4, 4]
 - Predicted Box: [3, 3, 4, 4]

Object Detection

Metrics

Scenario 1:

- Ground Truth Box: $[2, 2, 4, 4] \rightarrow (2,2) \text{ to } (6,6)$
- Predicted Box: $[4, 4, 4, 4] \rightarrow (4,4) \text{ to } (8,8)$

Calculate Intersection Box:

- Top-left corner: $\max(2, 4), \max(2, 4) \rightarrow (4,4)$
- Bottom-right corner: $\min(6, 8), \min(6, 8) \rightarrow (6,6)$

Compute Intersection Area:

So the intersection box is: $(4, 4) \text{ to } (6, 6)$

- Width of the intersection: $6 - 4 = 2$
- Height of the intersection: $6 - 4 = 2$
- Area of the intersection: $2 \times 2 = 4$

Object Detection

Metrics

Compute Union Area:

- $Area_{GT} = (6 - 2) * (6 - 2) = 16$
- $Area_{Pred} = (8 - 4) * (8 - 4) = 16$
- $Area_{Union} = Area_{GT} + Area_{Pred} - Area_{Intersection} = 16 + 16 - 4 = 28$

Compute the IoU:

- $IoU = \frac{Area_{Intersection}}{Area_{Union}} = \frac{4}{28} = 0.1429$ (14.29% overlap)

Object Detection

Metrics

Scenario 2:

- Ground Truth Box: [2, 2, 4, 4]
- Predicted Box: [3, 3, 4, 4]

Object Detection

Metrics

Precision: Proportion of correct detected objects among all detections

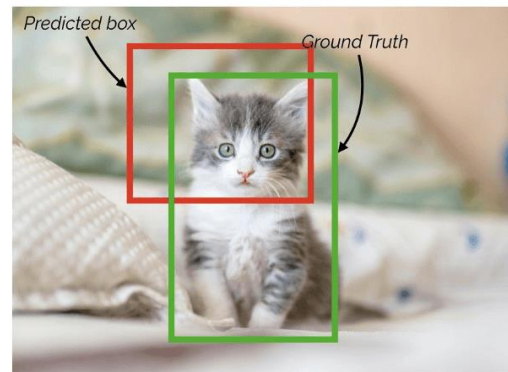
$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall: Proportion of correct detected objects among all actual objects

$$\text{Recall} = \frac{TP}{TP + FN}$$

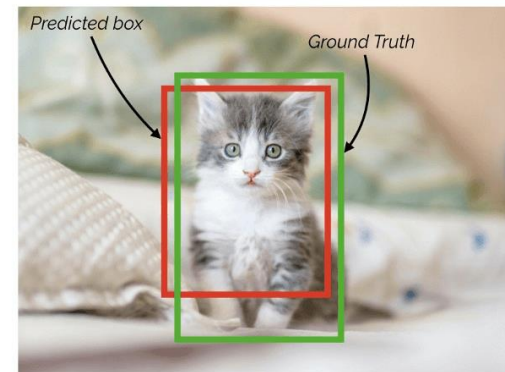
If IoU threshold = 0.5

False Positive (FP)



IoU = ~0.3

True Positive (TP)



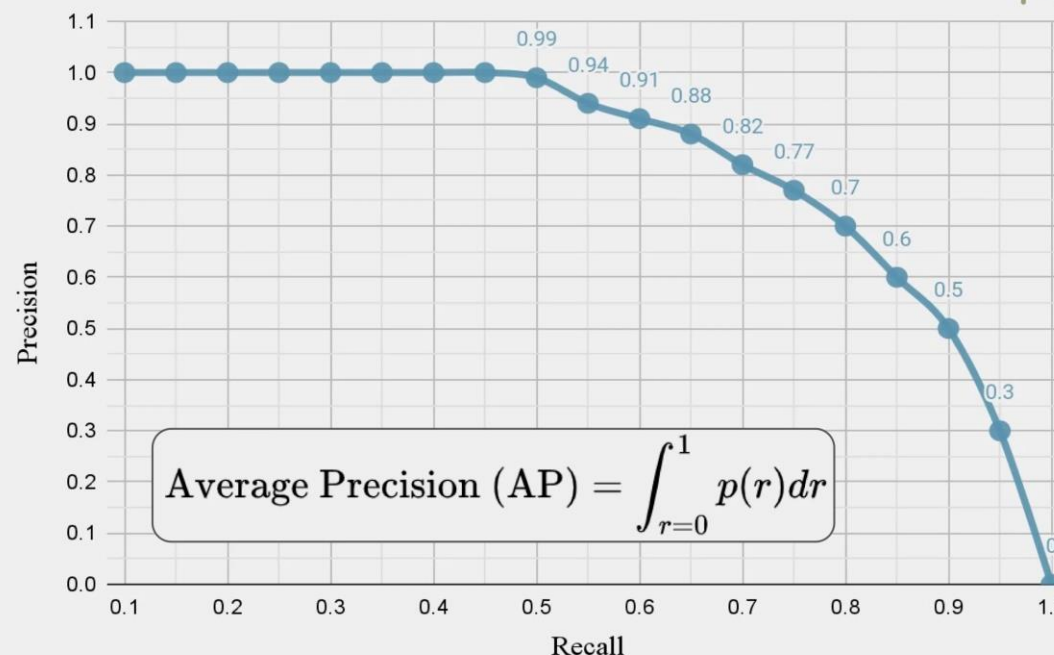
IoU = ~0.7

Object Detection Metrics

Precision Recall Curve (PR Curve)

Conf. Thresh.	Recall	Precision	Effect
0.95	0.1	1	More FN
0.9	0.15	1	
0.85	0.2	1	
0.8	0.25	1	
0.75	0.3	1	
0.7	0.35	1	
0.65	0.4	1	
0.6	0.45	1	
0.55	0.5	0.99	
0.5	0.55	0.94	
0.45	0.6	0.91	
0.4	0.65	0.88	
0.35	0.7	0.82	
0.3	0.75	0.77	
0.25	0.8	0.7	
0.2	0.85	0.6	
0.15	0.9	0.5	
0.1	0.95	0.3	
0.05	1	0	More FP

The **smaller the probability confidence threshold**, the higher the number of detections made by the model, and the lower the chances that the ground-truth labels were missed and hence **higher the recall** (*Generally, but not always*). On the other hand, **the higher the confidence threshold**, the more confident the model is in what it predicts and hence **higher the precision** (*Generally, but not always*).



Object Detection

Metrics

Mean Average Precision (mAP):

- The mean of the Average Precision (AP) values across all object classes.

$$\text{Mean Average Precision (mAP)} = \frac{1}{N} \sum_{i=1}^N AP_i$$

- AP measures the area under the precision-recall curve for one class.
- In COCO evaluation, mAP is additionally averaged over multiple IoU thresholds, typically from 0.50 to 0.95.

Object Localization Example

- Input image: $1 \times 28 \times 28$
 - One grayscale image with 28×28 pixels
- Conv1: $1 \rightarrow 32$ channels
 - Learns 32 filters and creates 32 feature maps
- ReLU activation
 - Keeps positive values and sets negative values to 0
- Max Pooling: 2×2 , stride 2
 - Reduces spatial size from 28×28 to 14×14
- Conv2: $32 \rightarrow 64$ channels
 - Learns more complex features from the first feature maps

Max Pooling again

- Reduces spatial size from 14×14 to 7×7

```
class BasicLocalizationCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        # Input: 1 x 28 x 28
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=32,
                                kernel_size=3, stride=1, padding=1)

        # Output: 32 x 28 x 28
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

        # After pooling: 32 x 14 x 14
        self.conv2 = nn.Conv2d(in_channels=32, out_channels=64,
                                kernel_size=3, stride=1, padding=1)

        # Output: 64 x 14 x 14
        # After pooling: 64 x 7 x 7
        # Shared fully connected feature layer
        self.fc1 = nn.Linear(64 * 7 * 7, 128)

        # Head 1: classification
        # Predicts class scores, e.g. digits 0-9
        self.class_head = nn.Linear(128, num_classes)

        # Head 2: bounding box regression
        # Predicts [x, y, width, height]
        self.bbox_head = nn.Linear(128, 4)
```

Object Localization Example

- Flatten
 - Converts $64 \times 7 \times 7$ feature maps into a vector of 3136 values
- Shared fully connected layer
 - Maps extracted visual features to a compact 128-dimensional representation
- Classification head
 - Predicts class scores for the object
 - Output: 10 logits, one score for each digit from 0 to 9
- Bounding box regression head
 - Predicts the object location
 - Output: 4 values: [x, y, width, height]
- Model output
 - class_logits: What object is in the image?
 - bbox: Where is the object located?

```
def forward(self, x):  
    x = self.conv1(x)          # Convolution  
    x = F.relu(x)             # ReLU activation  
    x = self.pool(x)          # Max pooling  
    x = self.conv2(x)          # Convolution  
    x = F.relu(x)             # ReLU activation  
    x = self.pool(x)          # Max pooling  
    x = torch.flatten(x, 1)    # Keep batch dimension, flatten the rest  
    x = self.fc1(x)            # Shared fully connected layer  
    x = F.relu(x)             # ReLU activation  
    class_logits = self.class_head(x) # Output: class scores  
    bbox = self.bbox_head(x)    # Output: bounding box coordinates  
    return class_logits, bbox
```

```
model = BasicLocalizationCNN(num_classes=10)  
classification_loss_fn = nn.CrossEntropyLoss()  
bbox_loss_fn = nn.SmoothL1Loss()  
images = torch.randn(64, 1, 28, 28)      # Example batch  
labels = torch.randint(0, 10, (64,))      # Class labels  
target_boxes = torch.rand(64, 4)          # [x, y, width, height]  
class_logits, pred_boxes = model(images)  
classification_loss = classification_loss_fn(class_logits, labels)  
bbox_loss = bbox_loss_fn(pred_boxes, target_boxes)  
loss = classification_loss + bbox_loss  
loss.backward()
```

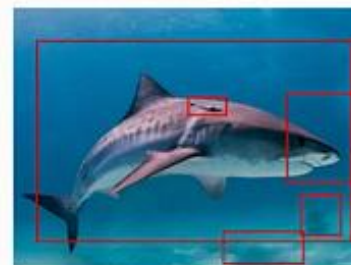
Object Detection

Types

One-Stage detector



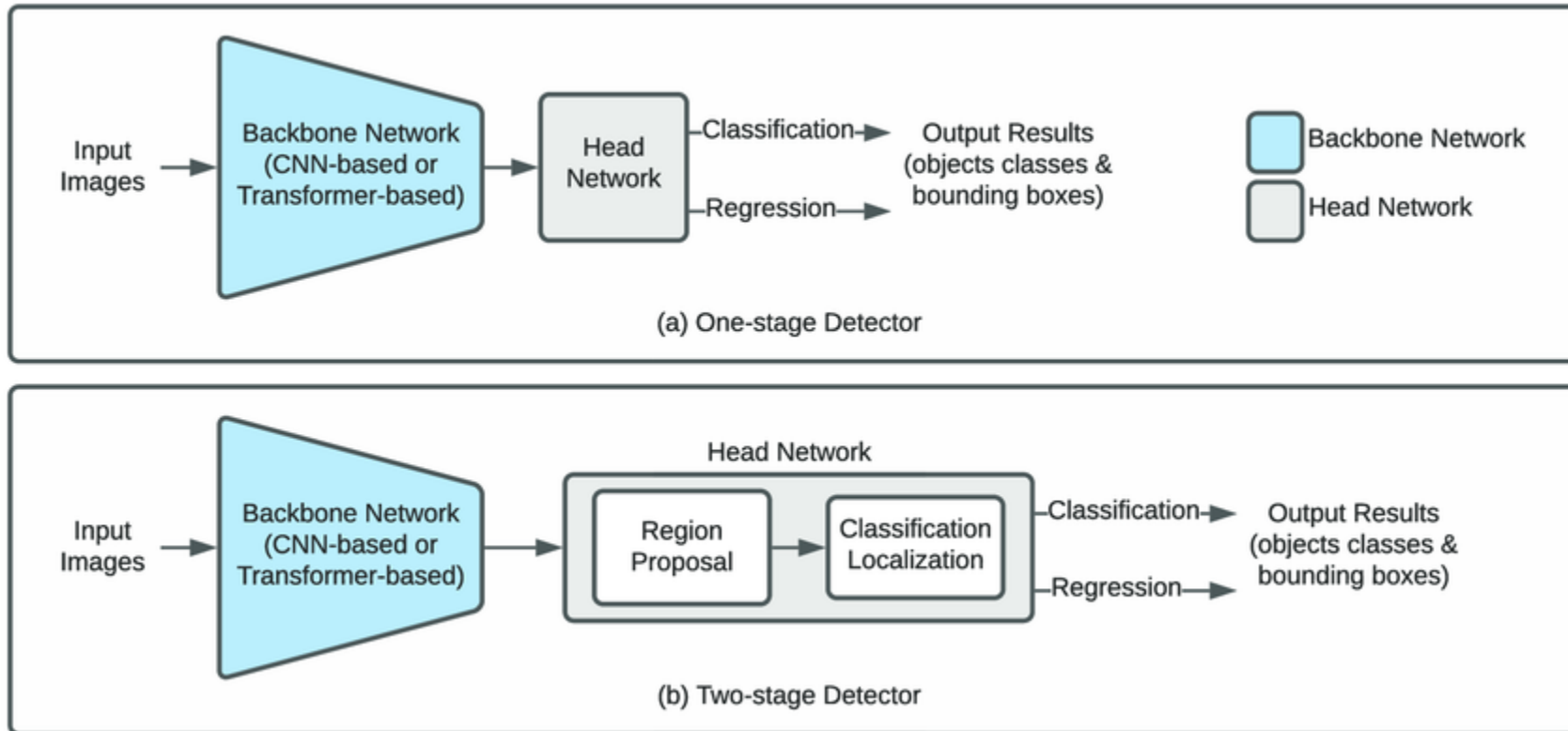
Two-Stage detector



region proposals

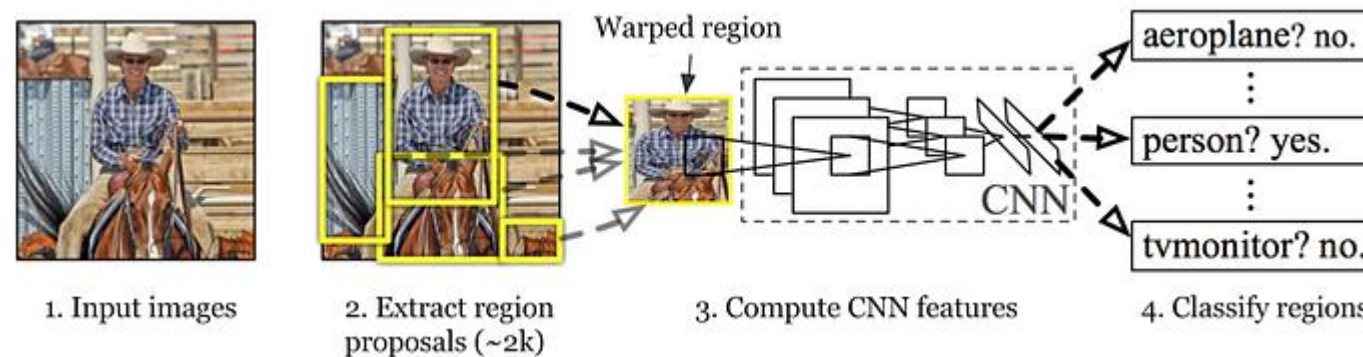


Object Detection Types



Two-Stage Detectors

- Stage 1: Region proposal generation
 - Model generates candidate regions (Region of Interests - Rols) likely to contain objects.
 - Done by a Region Proposal Network (RPN).
 - Goal: reduce search space, not classify yet.
- Stage 2: Object classification and box refinement
 - Each ROI gets classified (classification head) + box adjusted (regression head)
 - Refines location and assigns label



Faster R-CNN

R-CNN (2014)

- Uses handcrafted region proposals via Selective Search
- Relies on low-level visual cues (color, texture, size)
- Not trainable and computationally expensive

Faster R-CNN (2016)

- Introduces a Region Proposal Network (RPN)
- Generates proposals directly from CNN feature maps
- Fully trainable, integrated end-to-end
- Enables faster and more accurate detection

Faster R-CNN

R-CNN Selective Search

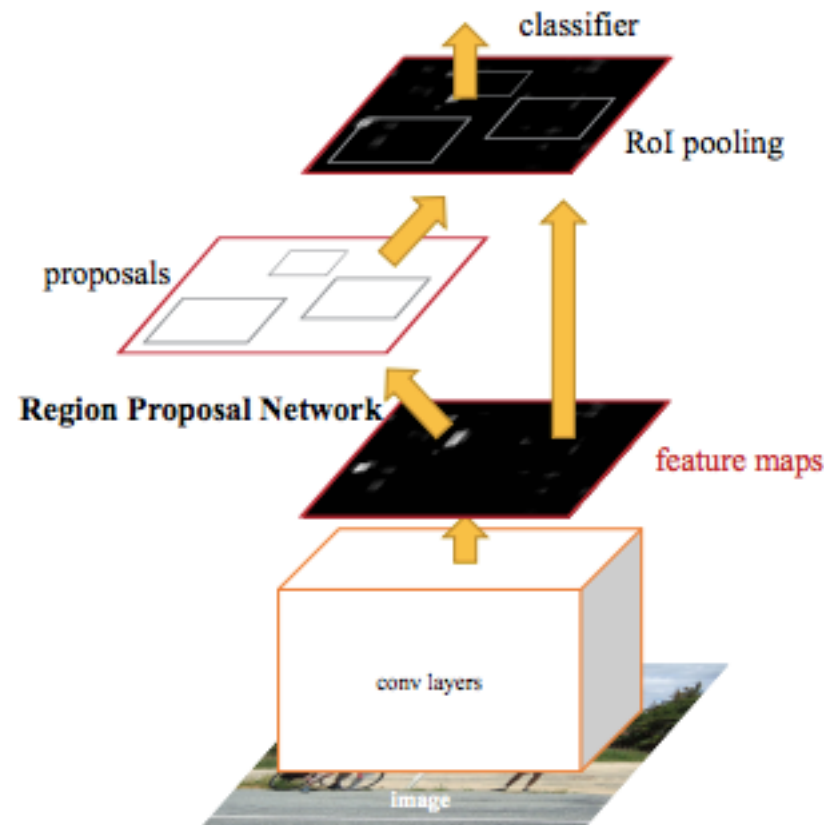
Combine exhaustive search (exploring many regions) with segmentation-based grouping (merging similar regions).

- Process:
 - Initial Segmentation: split image into small regions based on color, texture, and intensity.
 - Hierarchical Merging: iteratively combine similar neighboring regions.
 - Proposal Extraction: each merged region becomes a bounding box (RoI) candidate.
- Output:
 - ~2,000 region proposals per image.
 - Each proposal passed through CNN for feature extraction and classification.
- Limitations:
 - Slow (Handcrafted).
 - Not learnable, can't improve with training.

Faster R-CNN

Stage 1: Region Proposal Network

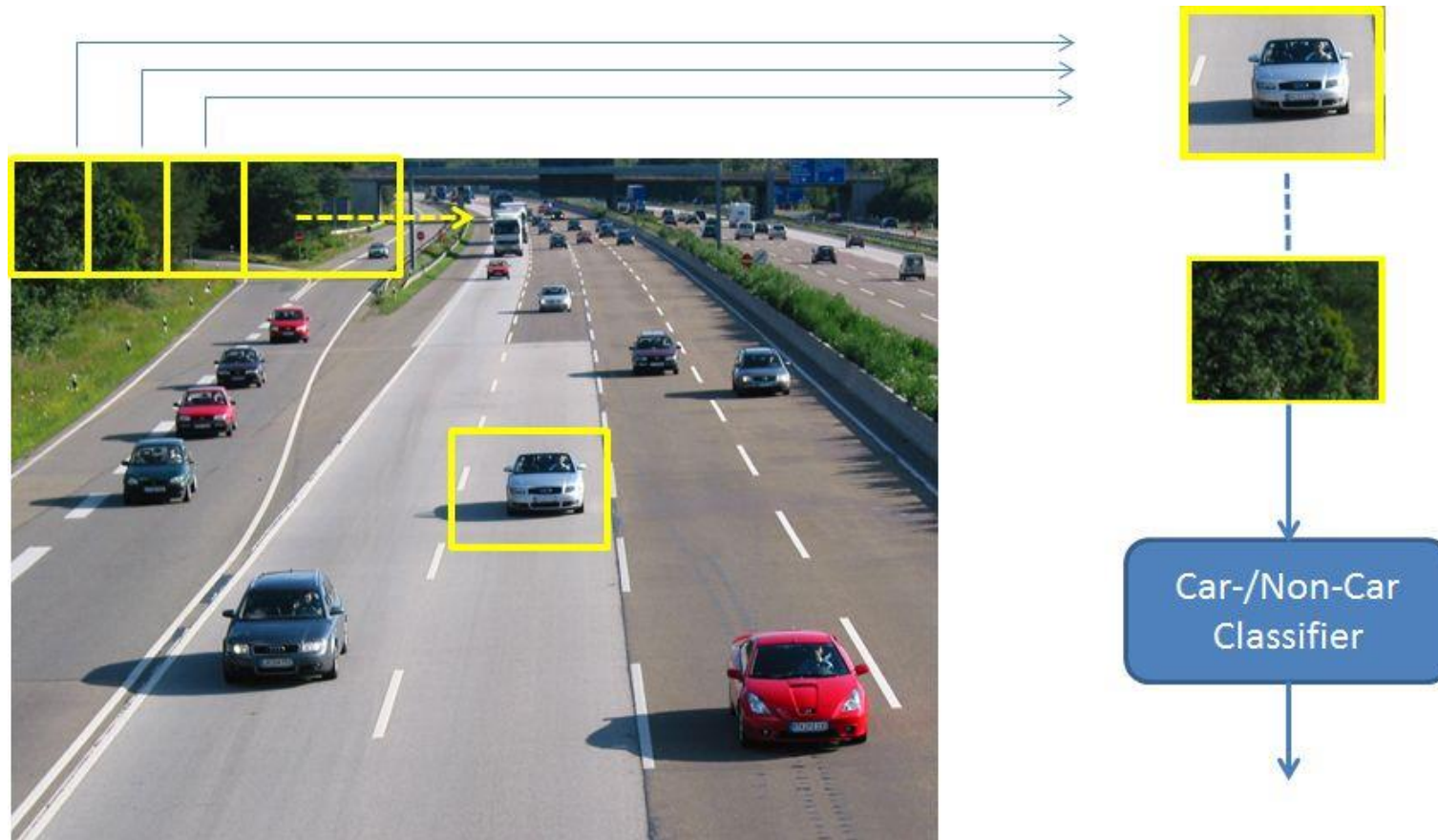
Small network slides over the final convolutional feature map to generate region proposals that indicate likely object locations (Faster R-CNN image).



Faster R-CNN

Stage 1: Region Proposal Network

Window slides over entire image. For each position, a classifier determines which object is contained in the window.



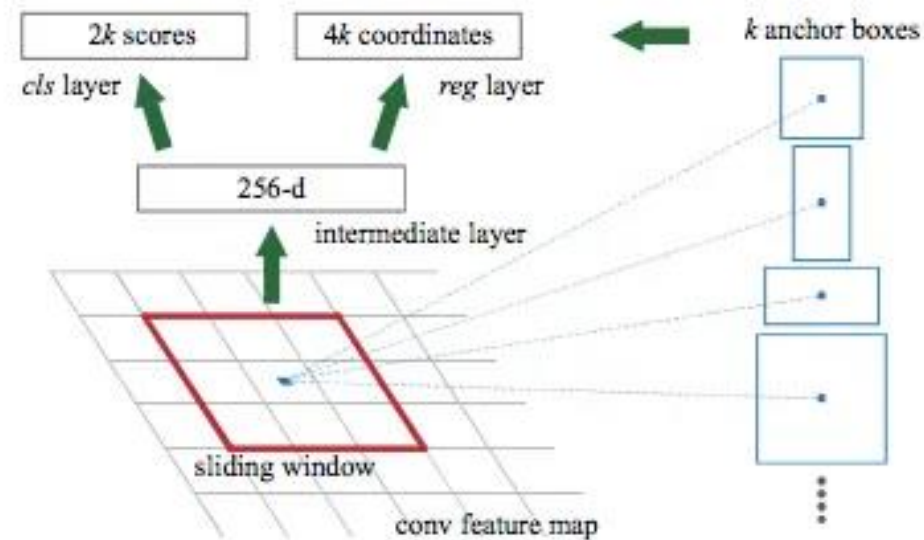
Faster R-CNN

Stage 1: Region Proposal Network

RPN operates on feature map of the CNN backbone.

Includes a classifier ($2k$) for probability of region being object or background and regressor ($4k$) for bounding box coordinates.

A window slides over feature map and generates proposals based on anchor boxes.



Faster R-CNN

Stage 1: Region Proposal Network

- Each position on the feature map generates multiple anchor boxes with different scales and aspect ratios (e.g., $3 \times 3 = 9$ anchors).
- Anchors act as reference boxes from which the network predicts object locations.

Anchor Box 1



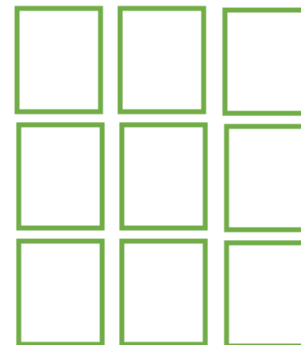
Anchor Box 1 tiled across image



Anchor Box 2



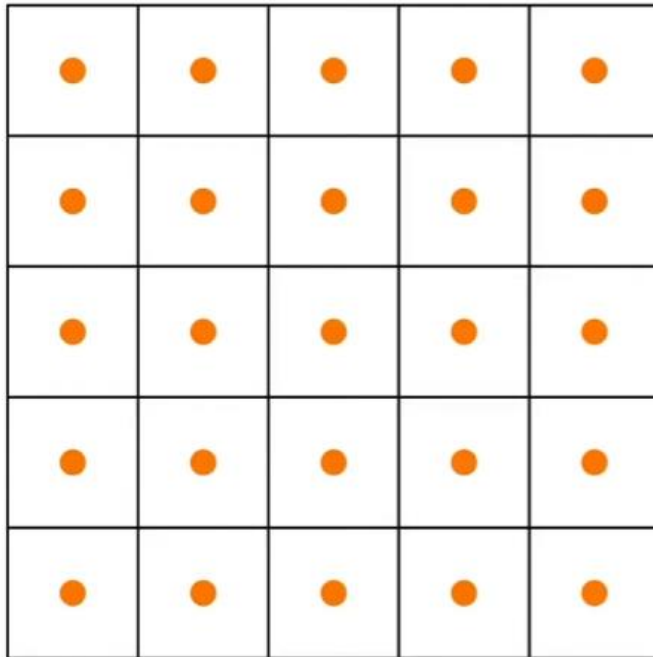
Anchor Box 2 tiled across image



Faster R-CNN

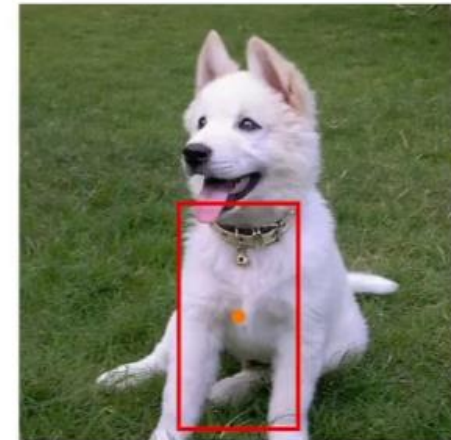
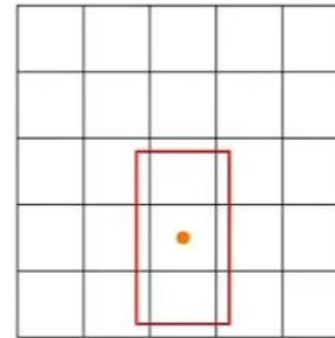
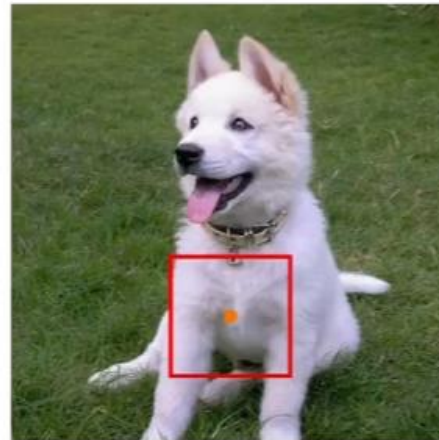
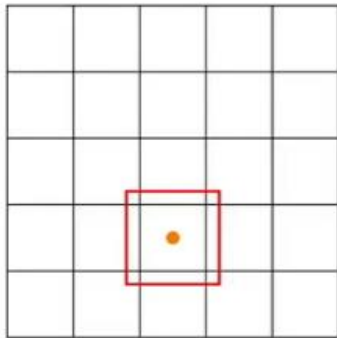
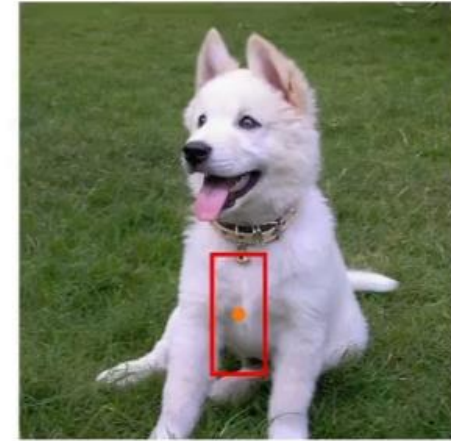
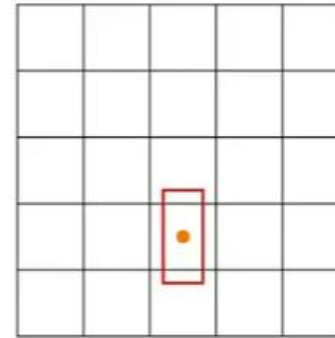
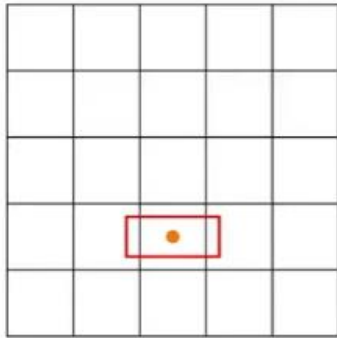
Stage 1: Region Proposal Network

Feature map anchor points



Faster R-CNN

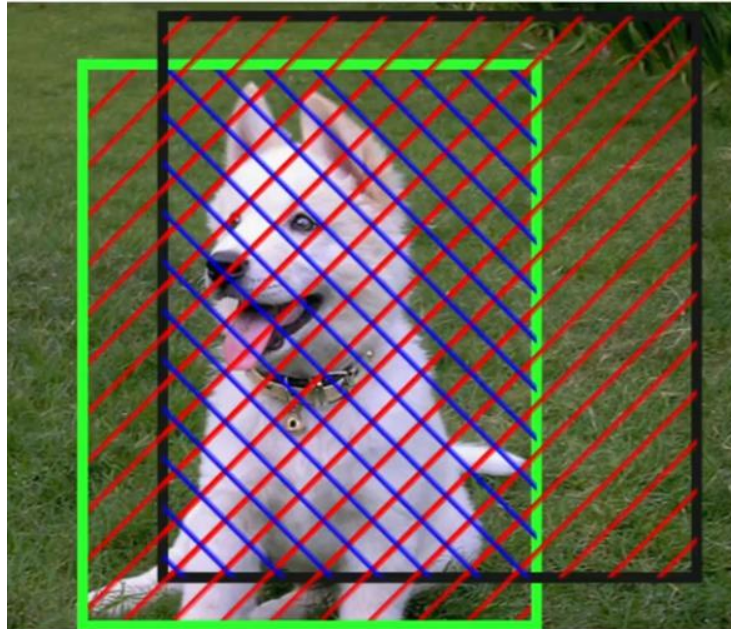
Stage 1: Region Proposal Network

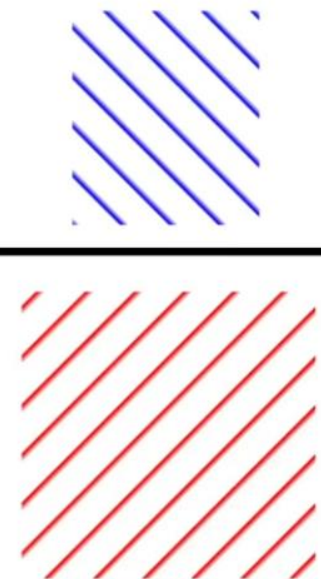


Faster R-CNN

Stage 1: Region Proposal Network

Returns region proposals based on IoU (refined boxes that likely contain objects).



$$\text{IoU} = \frac{\text{Area of Intersection}}{\text{Area of Union}}$$
A diagram illustrating the Intersection over Union (IoU) formula. It shows two overlapping rectangles: a blue one on top and a red one on the bottom. The intersection of the two rectangles is shaded with blue diagonal lines, and the union of the two rectangles is shaded with red diagonal lines. The formula $\text{IoU} = \frac{\text{Area of Intersection}}{\text{Area of Union}}$ is written to the left of the diagram.

Faster R-CNN

Stage 2: Object Classification and Box Refinement

- Problem:
 - Region proposals have different sizes, but neural network needs fixed-size inputs.
→ ROI Pooling resizes all region proposals to fixed size through pooling.
- Object Classification:
 - Each region proposal is passed to a network to predict the category of the object inside.
- Bounding Box Refinement:
 - The region proposals are refined again to better match the actual objects.

→ Using a Multi-Task Loss to optimize both steps (lambda to prioritize tasks)

$$L = L_{cls} + \lambda L_{reg}$$

Faster R-CNN Example

- Load a model pretrained on COCO
- Keep the CNN backbone and Region Proposal Network
- Replace the final prediction head
- Train the new head on the custom dataset
- Model output:
 - bounding boxes
 - class labels
 - confidence scores

```
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor,  
    fasterrcnn_resnet50_fpn  
  
num_classes = 4 # background + 3 object classes  
  
# Load pretrained Faster R-CNN  
model = fasterrcnn_resnet50_fpn(weights="DEFAULT,")  
  
# Get number of input features for the prediction head  
in_features = model.roi_heads.box_predictor.cls_score.in_features  
  
# Replace the prediction head for our own classes  
model.roi_heads.box_predictor = FastRCNNPredictor(in_features,  
    num_classes)
```

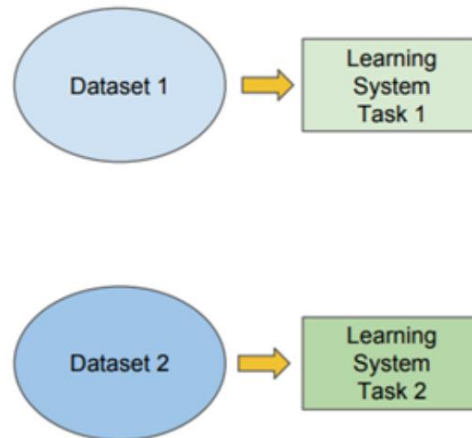
Transfer Learning

Traditional ML

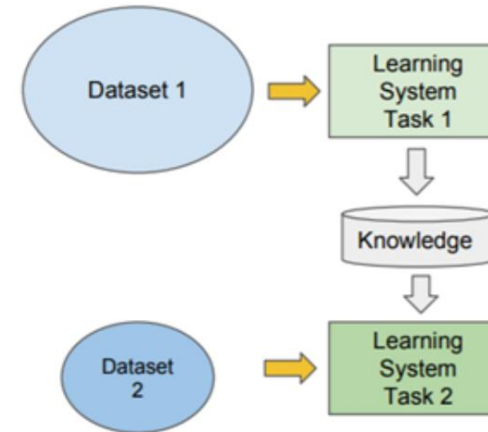
vs

Transfer Learning

- Isolated, single task learning:
 - Knowledge is not retained or accumulated. Learning is performed w.o. considering past learned knowledge in other tasks



- Learning of a new tasks relies on the previous learned tasks:
 - Learning process can be faster, more accurate and/or need less training data



Traditional Learning vs Transfer Learning

Transfer Learning

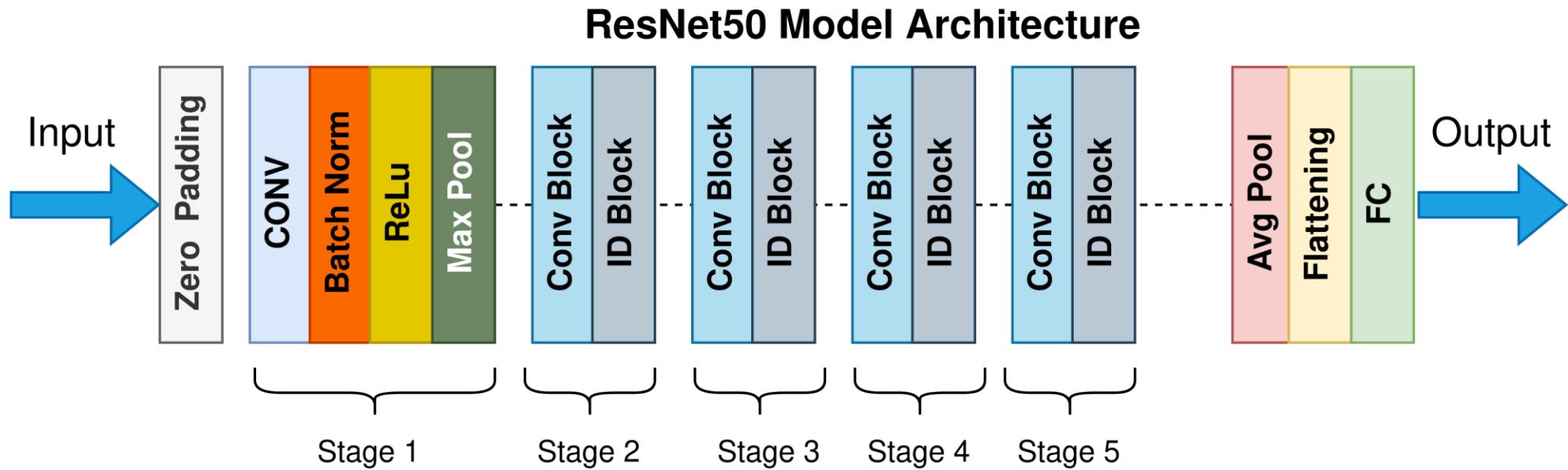
Instead of training from random weights on your specific dataset, you start with a model that already understands general visual features (edges, textures, shapes, object parts) from pre-training on a large dataset like ImageNet, COCO, or Objects365.

Model	Properties
VGG	Historical baseline, easy to understand
ResNet	Standard transfer-learning backbone
MobileNet	Lightweight / mobile-friendly
EfficientNet	Accuracy-efficiency trade-off
ConvNeXt	Modern CNN baseline

Overview:

<https://docs.pytorch.org/vision/main/models.html>

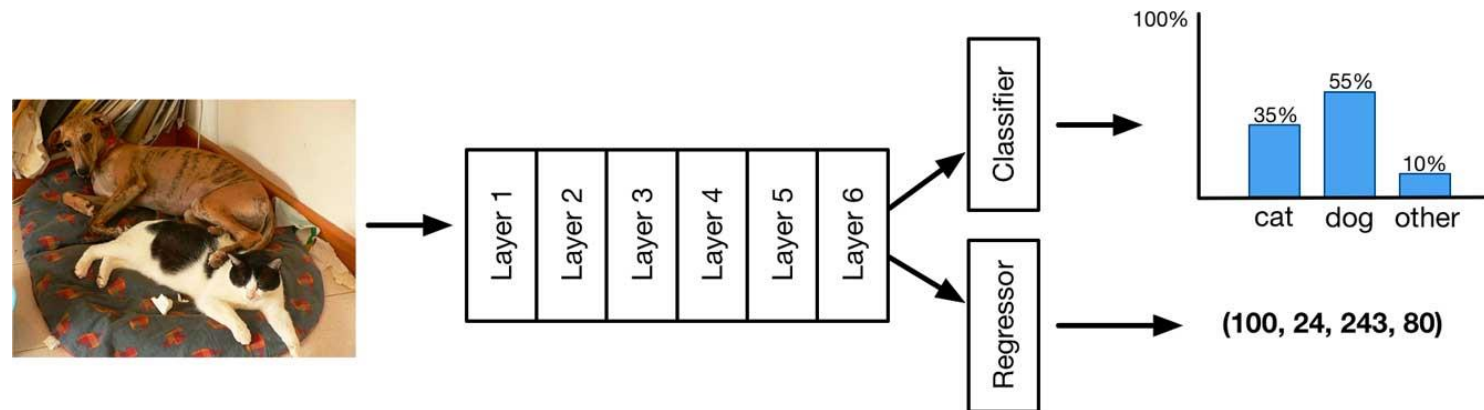
Transfer Learning



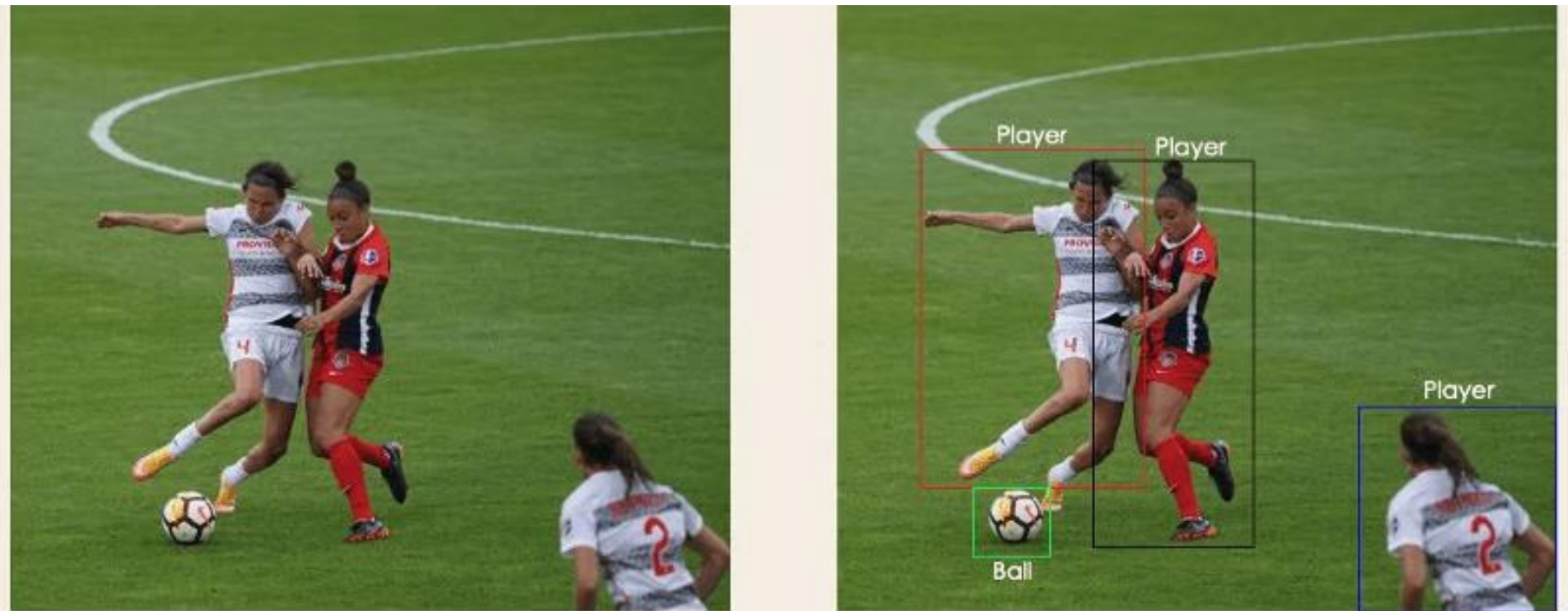
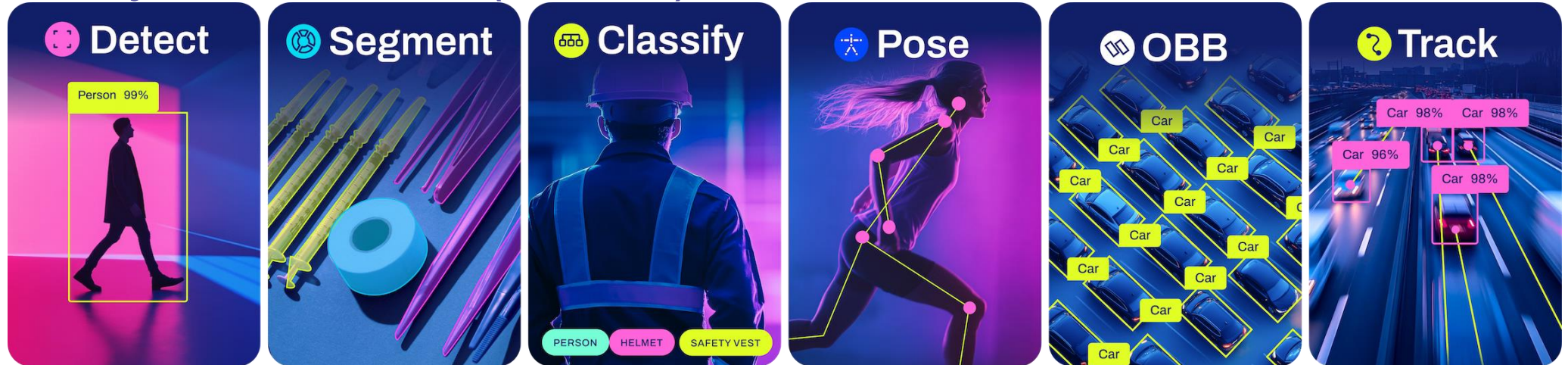
One-Stage Detector

Entire image processes through one forward pass through a single CNN to extract multi-scale feature maps.

Detection head predicts bounding boxes, class scores and confidence values.

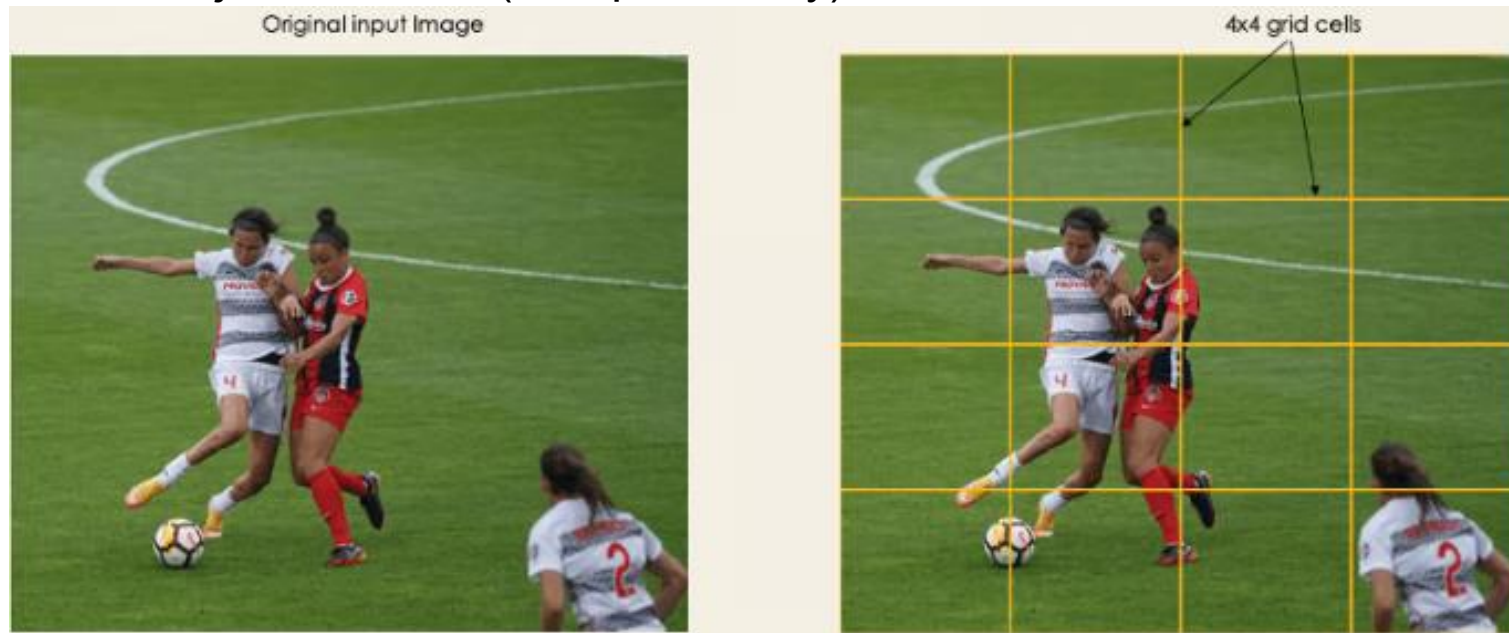


You Only Look Once (YOLO)



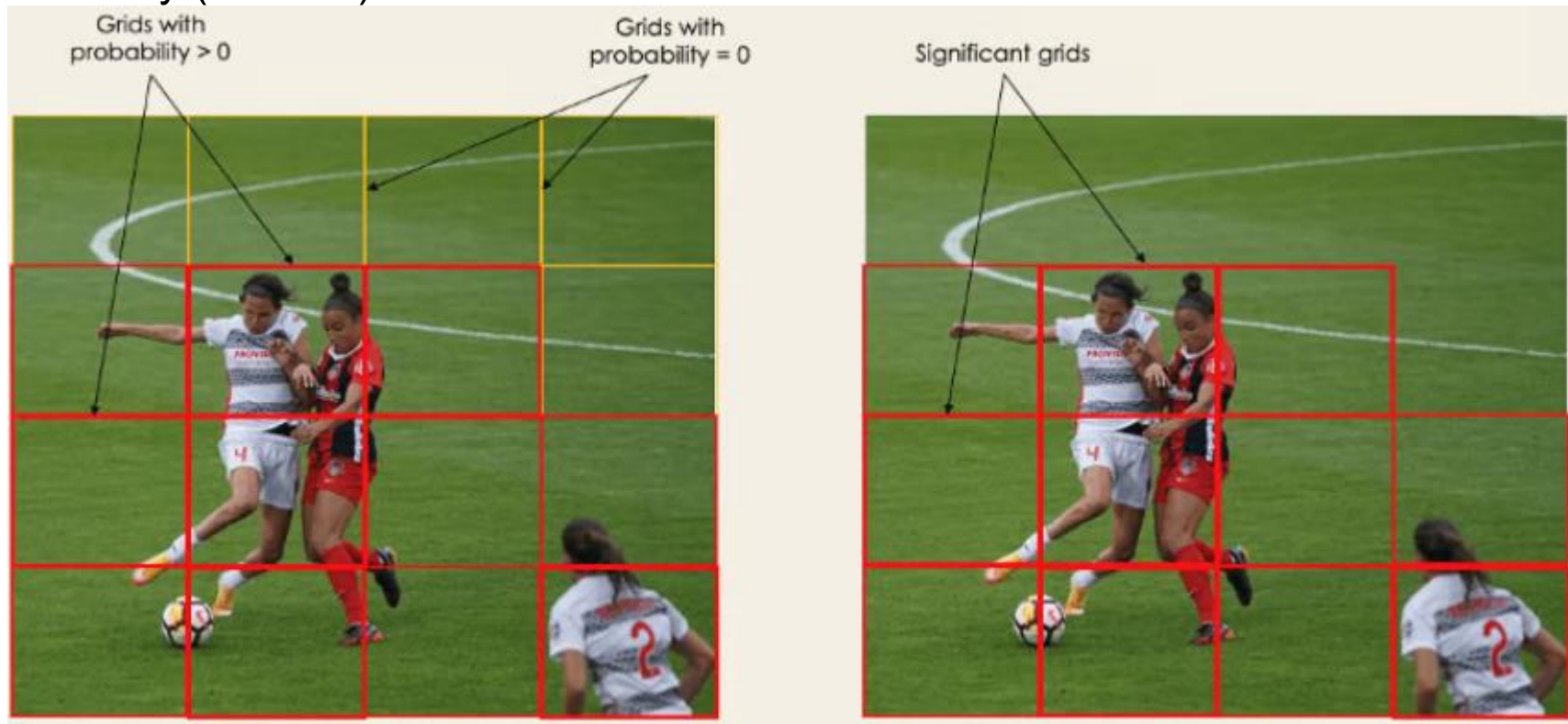
You Only Look Once (YOLO)

- Steps:
 1. Divide Image into a grid ($S \times S$). Each cell is responsible for localizing and predicting the class of the object it covers (incl. probability)



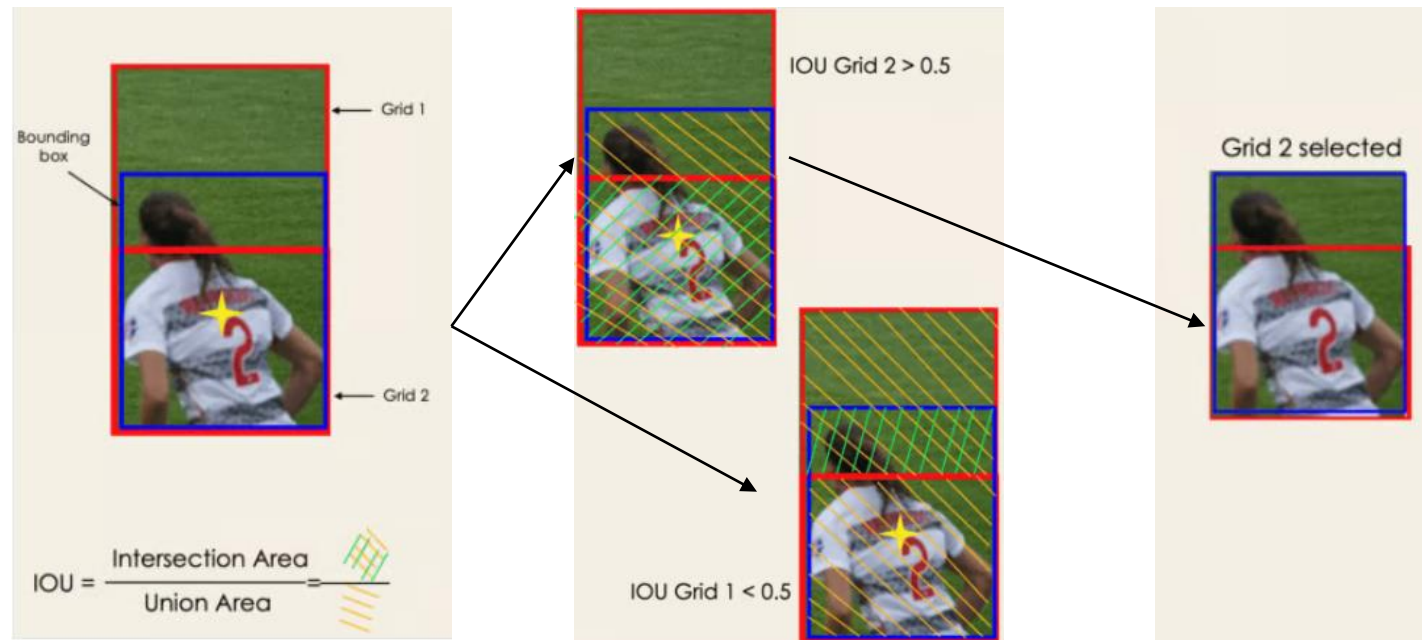
You Only Look Once (YOLO)

- Steps:
 2. Determine bounding boxes corresponding to rectangles, highlighting all objects in image. YOLO uses single regression module with existence probability, box coordinates and class probability (one-hot).



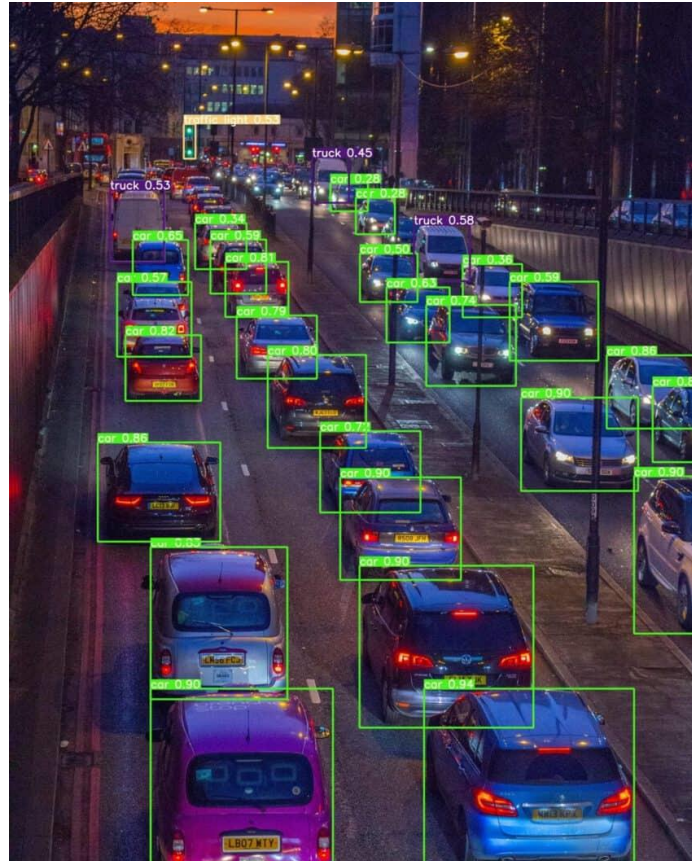
You Only Look Once (YOLO)

- Steps:
 3. Single object often has multiple grid box candidates. Take best based on IoU threshold (0.5 common).



You Only Look Once (YOLO)

- Steps:
 1. Bounding box regression. Adjust the bounding box to better fit the object.
 2. Confidence score. Measure the probability that the object is present in the image.
 3. Non-Max suppression. Remove overlapping boxes that are less likely to be the object.
 4. Non-Max suppression. Keep only boxes with highest probably detection score.



You Only Look Once (YOLO) - Example

- Load a pretrained YOLO model
- Train it on a custom dataset using YOLO format labels
- The model directly predicts:
 - bounding boxes
 - class labels
 - confidence scores

```
from ultralytics import YOLO

# Load a pretrained YOLO model
# "n" = nano version, small and fast
model = YOLO("yolo26n.pt")

# Train the model on a custom dataset
results = model.train(data="data.yaml", epochs=50, imgsz=640, batch=16)

# Validate the trained model
metrics = model.val()

# Run inference on a new image
predictions = model.predict(source="test_image.jpg", conf=0.25)
```

data.yaml:

```
path: /path/to/dataset
train: images/train
val: images/val
```

names:

```
0: helmet
1: vest
2: worker
```